Informatica

Informatica® PowerExchange for HDFS
10.5.10

User Guide

Informatica PowerExchange for HDFS User Guide
10.5.10
February 2026

© Copyright Informatica LLC 2012, 2025

This software and documentation are provided only under a separate license agreement containing restrictions on use and disclosure. No part of this document may be reproduced or transmitted in any form, by any means (electronic, photocopying, recording or otherwise) without prior consent of Informatica LLC.

U.S. GOVERNMENT RIGHTS Programs, software, databases, and related documentation and technical data delivered to U.S. Government customers are "commercial computer software" or "commercial technical data" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, duplication, disclosure, modification, and adaptation is subject to the restrictions and license terms set forth in the applicable Government contract, and, to the extent applicable by the terms of the Government contract, the additional rights set forth in FAR 52.227-19, Commercial Computer Software License.

Informatica, the Informatica logo, and PowerExchange are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html. Other company and product names may be trade names or trademarks of their respective owners.

Portions of this software and/or documentation are subject to copyright held by third parties. Required third party notices are included with the product.

The information in this documentation is subject to change without notice. If you find any problems in this documentation, report them to us at infa_documentation@informatica.com.

Informatica products are warranted according to the terms and conditions of the agreements under which they are provided. INFORMATICA PROVIDES THE INFORMATION IN THIS DOCUMENT "AS IS" WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING WITHOUT ANY WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND ANY WARRANTY OR CONDITION OF NON-INFRINGEMENT.

Publication Date: 2026-02-19

Table of Contents

Preface 6

Informatica Resources 6

- Informatica Network 6
- Informatica Knowledge Base 6
- Informatica Documentation 6
- Informatica Product Availability Matrices 7
- Informatica Velocity 7
- Informatica Marketplace 7
- Informatica Global Customer Support 7

Chapter 1: Introduction to PowerExchange for HDFS 8

PowerExchange for HDFS Overview 8

Chapter 2: PowerExchange for HDFS Configuration 9

PowerExchange for HDFS Configuration Overview 9

Prerequisites 9

Chapter 3: HDFS Connections 10

HDFS or View File System (ViewFS) Connections Overview 10

HDFS or View File System (ViewFS) Connection Properties 10

Creating an HDFS or View File System (ViewFS) Connection 12

Chapter 4: HDFS Data Objects 13

HDFS or View File System (ViewFS) Data Objects Overview 13

Generate the Source File Name for HDFS or View File System (ViewFS) Data Objects 13

FileName Port Overview 14

- Working with FileName Port 14
- Rules and Guidelines for Using FileName Port 14

Flat File Data Objects 16

- Compression and Decompression for Flat File Sources and Targets 17
- Rules and Guidelines for Flat File Data Objects 17
- Configuring a Flat File Data Object with an HDFS Connection 18
- Naming Convention for Flat File Targets 18

Complex File Data Objects 18

- Complex File Data Object Overview Properties 19
- Compression and Decompression for Complex File Sources and Targets 20
- Parameterization of Complex File Data Objects 20
- Complex File Data Object Output Parsing 21
- Creating a Complex File Data Object 21
- Creating a Complex File Object Read or Write Operation 23

Rules and Guidelines for Creating a Complex File Data Object Operation. 23
Custom Formats. 24
Custom Formats Configuration. 24

Chapter 5: HDFS Data Extraction. 25

HDFS or View File System (ViewFS) Data Extraction Overview. 25
Flat File Data Object Read Properties. 25
Complex Files Partitioning. 26
Complex File Data Object Read Properties. 26
Wildcard Characters for Reading Data from Complex Files. 27
General Properties. 27
Ports Properties. 28
Schema Properties. 28
Sources Properties. 29
Advanced Properties. 30

Chapter 6: HDFS Data Load. 32

HDFS or View File System (ViewFS) Data Load Overview. 32
Flat File Data Object Write Properties. 32
Complex File Streaming. 33
Complex Files Output Collection Mode. 34
Complex File Data Object Write Properties. 35
Overwriting Complex File Targets. 35
General Properties. 36
Port Properties. 36
Schema Properties. 36
Target Properties. 37
Advanced Properties. 38

Chapter 7: HDFS Mappings. 40

HDFS or View File System (ViewFS) Mappings Overview. 40
Complex Files Target Creation. 41
Directory-Level Partitioning. 41
Rules and Guidelines for Reading from or Writing to Partition Columns. 45
Creating a Complex File Target from an Existing Transformation 45
Mapping Validation and Run-time Environments. 46
Audits. 46
Rules and Guidelines for Complex File Sources and Targets in a Mapping. 46
HDFS or View File System (ViewFS) Data Extraction Mapping Example. 47
HDFS or View File System (ViewFS) Data Load Mapping Example. 48
HDFS or View File System (ViewFS) Avro Read Mapping Example. 50

Appendix A: Data Type Reference. 53

Data Type Reference Overview. 53

Flat File and Transformation Data Types. 54

Complex File and Transformation Data Types. 55

Avro Data Types and Transformation Data Types. 55

JSON Data Types and Transformation Data Types. 57

ORC Data Types and Transformation Data Types. 57

Parquet Data Types and Transformation Data Types. 59

Rules and Guidelines for Data Types. 60

Index. 62

Preface

Use the *Informatica® PowerExchange® for HDFS User Guide* to learn how to read from or write to Hadoop Distributed File System by using the Developer tool. Learn to create a connection, develop and run mappings and dynamic mappings in the native environment and Hadoop environments.

Informatica Resources

Informatica provides you with a range of product resources through the Informatica Network and other online portals. Use the resources to get the most from your Informatica products and solutions and to learn from other Informatica users and subject matter experts.

Informatica Network

The Informatica Network is the gateway to many resources, including the Informatica Knowledge Base and Informatica Global Customer Support. To enter the Informatica Network, visit https://network.informatica.com.

As an Informatica Network member, you have the following options:

- Search the Knowledge Base for product resources.
- View product availability information.
- Create and review your support cases.
- Find your local Informatica User Group Network and collaborate with your peers.

Informatica Knowledge Base

Use the Informatica Knowledge Base to find product resources such as how-to articles, best practices, video tutorials, and answers to frequently asked questions.

To search the Knowledge Base, visit https://search.informatica.com. If you have questions, comments, or ideas about the Knowledge Base, contact the Informatica Knowledge Base team at KB_Feedback@informatica.com.

Informatica Documentation

Use the Informatica Documentation Portal to explore an extensive library of documentation for current and recent product releases. To explore the Documentation Portal, visit https://docs.informatica.com.

If you have questions, comments, or ideas about the product documentation, contact the Informatica Documentation team at infa_documentation@informatica.com.

Informatica Product Availability Matrices

Product Availability Matrices (PAMs) indicate the versions of the operating systems, databases, and types of data sources and targets that a product release supports. You can browse the Informatica PAMs at https://network.informatica.com/community/informatica-network/product-availability-matrices.

Informatica Velocity

Informatica Velocity is a collection of tips and best practices developed by Informatica Professional Services and based on real-world experiences from hundreds of data management projects. Informatica Velocity represents the collective knowledge of Informatica consultants who work with organizations around the world to plan, develop, deploy, and maintain successful data management solutions.

You can find Informatica Velocity resources at http://velocity.informatica.com. If you have questions, comments, or ideas about Informatica Velocity, contact Informatica Professional Services at ips@informatica.com.

Informatica Marketplace

The Informatica Marketplace is a forum where you can find solutions that extend and enhance your Informatica implementations. Leverage any of the hundreds of solutions from Informatica developers and partners on the Marketplace to improve your productivity and speed up time to implementation on your projects. You can find the Informatica Marketplace at https://marketplace.informatica.com.

Informatica Global Customer Support

You can contact a Global Support Center by telephone or through the Informatica Network.

To find your local Informatica Global Customer Support telephone number, visit the Informatica website at the following link:
https://www.informatica.com/services-and-training/customer-success-services/contact-us.html.

To find online support resources on the Informatica Network, visit https://network.informatica.com and select the eSupport option.

CHAPTER 1

Introduction to PowerExchange for HDFS

This chapter includes the following topic:

- PowerExchange for HDFS Overview, 8

PowerExchange for HDFS Overview

PowerExchange for HDFS provides connectivity to the Hadoop Distributed File System (HDFS) or View File System (ViewFS). You can use PowerExchange for HDFS to read data from and write data to HDFS or ViewFS. You can also use PowerExchange for HDFS to read data from and write data to the local file system.

The Data Integration Service uses the Hadoop API infrastructure to connect to HDFS or ViewFS. It connects to the NameNode to read data from and write data to HDFS or ViewFS.

With PowerExchange for HDFS, you can read and write fixed-width text files, delimited text files, and the industry-standard file formats, such as Avro, Parquet, ORC, JSON, and XML files. You can read and write hierarchical data present in the Avro, Parquet, ORC, JSON, and XML files. In addition to the industry-standard file formats, you can also read from intelligent structure sources.

With PowerExchange for HDFS, you can read and write files as binary and write them to a target. When you select the file type as binary for a complex file source or target, PowerExchange for HDFS can process the mapping with or without using the Data Processor transformation.

You can read and write compressed files. You can configure custom formats to process data in input, output, and compression formats that Hadoop supports.

CHAPTER 2

PowerExchange for HDFS Configuration

This chapter includes the following topics:

- PowerExchange for HDFS Configuration Overview, 9
- Prerequisites, 9

PowerExchange for HDFS Configuration Overview

PowerExchange for HDFS is installed with Informatica Data Services. You enable PowerExchange for HDFS with a license key.
Note: To read or write data with a complex file data object, you will also need the Unstructured Data license key.

Prerequisites

Before you use PowerExchange for HDFS to access data in HDFS or View File System (ViewFS), perform the following tasks:

- Install and configure Informatica Services. Verify that the domain has a Data Integration Service and a Model Repository Service.
- Verify that a cluster configuration is created in the domain.
- Verify that a Metadata Access Service is created in the domain.
- Verify that the Hadoop Distribution Directory property in the developerCore.ini file is set based on the Hadoop distribution that you use.
- To run a mapping to process complex files, you must configure the INFA_PARSER_HOME environment variable for the Data Integration Service in Informatica Administrator. Set the value of the environment variable to the absolute path of the Hadoop distribution directory on the machine that runs the Data Integration Service.

CHAPTER 3

HDFS Connections

This chapter includes the following topics:

- HDFS or View File System (ViewFS) Connections Overview, 10
- HDFS or View File System (ViewFS) Connection Properties, 10
- Creating an HDFS or View File System (ViewFS) Connection, 12

HDFS or View File System (ViewFS) Connections Overview

Create an HDFS connection to read data from or write data to HDFS or View File System (ViewFS).

HDFS or View File System (ViewFS) Connection Properties

Use a Hadoop File System (HDFS) or View File System (ViewFS) connection to access data in the Hadoop cluster. The HDFS or ViewFS connection is a file system type connection. You can create and manage an HDFS or ViewFS connection in the Administrator tool, Analyst tool, or the Developer tool. HDFS or ViewFS connection properties are case sensitive unless otherwise noted.

Note: The order of the connection properties might vary depending on the tool where you view them.

The following table describes HDFS connection properties:

Property	Description
Name	Name of the connection. The name is not case sensitive and must be unique within the domain. The name cannot exceed 128 characters, contain spaces, or contain the following special characters: ~ ` ! $ % ^ & * ( ) - + = { [}] | \ : ; " ' < , > . ? /
ID	String that the Data Integration Service uses to identify the connection. The ID is not case sensitive. It must be 255 characters or less and must be unique in the domain. You cannot change this property after you create the connection. Default value is the connection name.

Property	Description
Description	The description of the connection. The description cannot exceed 765 characters.
Location	The domain where you want to create the connection. Not valid for the Analyst tool.
Type	The connection type. Default is Hadoop File System.
User Name	User name to access HDFS.
NameNode URI	The URI to access the storage system. You can find the value for `fs.defaultFS` in the `core-site.xml` configuration set of the cluster configuration. If you create connections when you import the cluster configuration, the NameNode URI property is populated by default, and it is updated each time you refresh the cluster configuration. If you use a Cloudera CDP Public Cloud compute cluster and the HDFS is on a Cloudera Data Lake cluster, set the property *spark.yarn.access.hadoopFileSystems* in the Spark properties of the Hadoop Connection to the same value as set here.

Accessing Multiple Storage Types

Use the NameNode URI property in the connection parameters to connect to various storage types. The following table lists the storage type and the NameNode URI format for the storage type:

Storage	NameNode URI Format
HDFS	`hdfs://<namenode>:<port>` where: - `<namenode>` is the host name or IP address of the NameNode. - `<port>` is the port that the NameNode listens for remote procedure calls (RPC). `hdfs://<nameservice>` in case of NameNode high availability.
View File System (ViewFS)	`viewfs://<clusterX>`, where `clusterX` is the mount table for the cluster.
MapR-FS	`maprfs:///`

WASB in HDInsight	`wasb://<container_name>@<account_name>.blob.core.windows.net/<path>`
	where:
	- `<container_name>` identifies a specific Azure Storage Blob container.
	Note: `<container_name>` is optional.
	- `<account_name>` identifies the Azure Storage Blob object.
	Example:
	`wasb://infabdmoffering1storage.blob.core.windows.net/infabdmoffering1cluster/mr-history`

ADLS in HDInsight	`adl://home`

When you create a cluster configuration from an Azure HDInsight cluster, the cluster configuration uses either ADLS or WASB as the primary storage. You cannot create a cluster configuration with ADLS or WASB as the secondary storage. You can edit the NameNode URI property in the HDFS or ViewFS connection to connect to a local HDFS or ViewFS location.

Creating an HDFS or View File System (ViewFS) Connection

Create an HDFS or View File System (ViewFS) connection before you import physical data objects.

1. Click **Window** > **Preferences**.
2. Select **Informatica** > **Connections**.
3. Expand the domain.
4. Select the connection type **File Systems** > **Hadoop File System**, and click **Add**.
5. Enter a connection name.
6. Optionally, enter a connection description.
7. Click **Next**.
8. Enter the user name to access HDFS or ViewFS.
9. Enter the NameNode URI to access HDFS or ViewFS based on the Hadoop distribution that you use.

   For instance, enter the following value for HDFS: `hdfs://<namenode>:<port>/`.

   For ViewFS, enter the following value: `viewfs://<cluster_name>`
10. Select the cluster configuration associated with the Hadoop environment.
11. Click **Test Connection**. If a default Metadata Access Service is not set, a message appears to configure the Metadata Access Service. Click **OK** and set one Metadata Access Service as default. After you set a default Metadata Access Service, the connection to HDFS is tested. If the Metadata Access Service does not exist, contact the Informatica administrator to create a new Metadata Access Service in the domain.
12. Click **Finish**.

Related Topics:

- "HDFS or View File System (ViewFS) Connection Properties" on page 10

CHAPTER 4

HDFS Data Objects

This chapter includes the following topics:

- HDFS or View File System (ViewFS) Data Objects Overview, 13
- Generate the Source File Name for HDFS or View File System (ViewFS) Data Objects, 13
- FileName Port Overview, 14
- Flat File Data Objects, 16
- Complex File Data Objects, 18
- Custom Formats, 24

HDFS or View File System (ViewFS) Data Objects Overview

After you configure an HDFS or ViewFS connection, create a physical data object to read data from or write data to HDFS or ViewFS.

Depending on the file format, you can configure the following types of physical data objects:

- Flat file data object. Create or import a flat file data object and configure an HDFS or ViewFS connection for the data object. Use the flat file data object to read or write fixed-width or delimited text files.
- Complex file data object. Import a complex file data object with an HDFS or ViewFS connection. Use the complex file data object to read or write structured, semi-structured, unstructured data. For example, Avro, Parquet, Orc, XML, and JSON files have structured or semi-structured data. Binary files, such as PDF and Microsoft Word have unstructured data. Complex file data objects can also read intelligent structure sources.

Generate the Source File Name for HDFS or View File System (ViewFS) Data Objects

You can add a file name column to the flat file data object. The file name column helps you to identify the source file that contains a particular record of data. You can configure the mapping with the file name

column for both flat file and complex file data objects. When you read data from HDFS or ViewFS, you can extract the fully qualified path of the source file.

You can configure the mapping to write the source file name to each source row when you add a File Name Column port in the Overview view. The File Name Column port contains the name and the fully qualified path for each source file. The File Name Column port is a string port with a default precision of 256 characters.

If the file or directory is in HDFS or ViewFS, enter the path without the node URI. For example,`/user/lib/testdir` specifies the location of a directory in HDFS or ViewFS. The path must not contain more than 512 characters.

When you use a file name column in a Read transformation, the file name column returns the value in the following format for HDFS:

```
hdfs://<host name>:<port>/<file name path>
```

For example, the file name column returns `hdfs://irldv:5008/hive/warehouse/ff.txt`, where the host name is irldv and the port is 5008.

FileName Port Overview

A FileName port is a string port with a default precision of 1024 characters that contains the source path of a file.

You cannot configure the FileName port. You can delete the FileName port if you do not want to read or write the data in the FileName. You cannot create a folder name which contains more than 255 characters.

When you create a data object read or write operation for all the complex files, the FileName port is displayed by default.

FileName port appears when you run a mapping in the native environment or on the Spark engine to read or write an Avro, ORC, JSON, Parquet, or binary(native) file.

Working with FileName Port

You can use the FileName port in a complex file target to dynamically redirect the rows based on the value received by FileName port.

When you run a mapping in the native environment to read or write a flat file using the FileName port, the Data Integration Service creates separate directories for each value in the FileName port in the following format:

```
<CFW FileName>/<CFW FileName>=<valueFromMappingFlow>
```

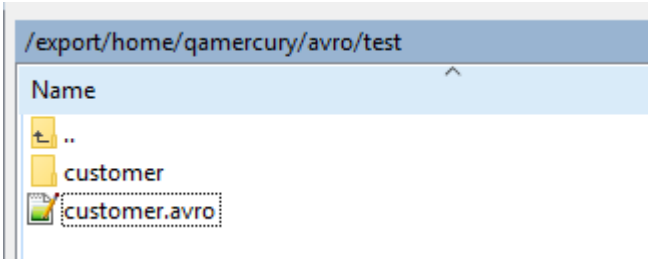
When you run a mapping in the native environment or on the Spark engine to read or write an Avro, JSON, ORC, Parquet or Binary(Native) file using the FileName port, the Data Integration Service creates separate directories for each value in the FileName port and adds the files within the directories.

Rules and Guidelines for Using FileName Port

Use the following rules and guidelines when you use the FileName data in the FileName port:

- Do not use a colon (:) and forward slash (/) character in the file name data of the FileName port of the source or target object to run a mapping.
- If you connect the FileName port to the target empty zero KB files are created in the target folder.

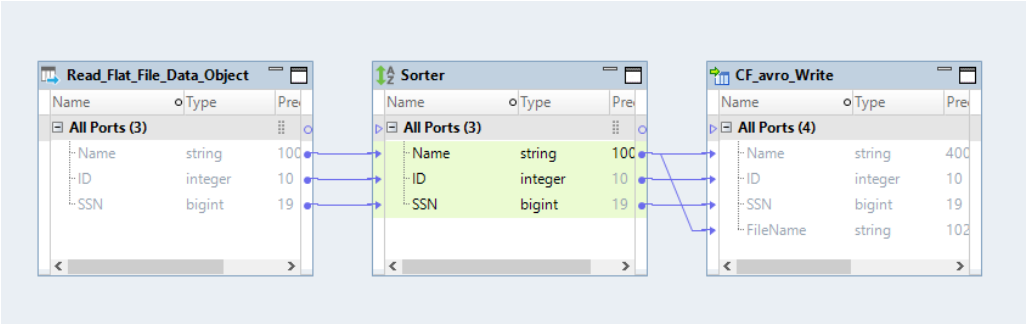
- When you use wildcard character * to read data from a complex file source, the Data Integration Service reads data only from folders or files matching the selection criteria.
  For example, if the file path is `/export/home/qamercury/avro/test/cust*` and **Allow Wildcard Characters** option is selected:



  The Data Integration Service ignores all the other folders and only reads `customer.avro` and the files present inside the customer folder.
- Do not connect FileName port to a FileName port because the FileName port in the source might contain colon (:) and forward slash (/) characters.
- In the Native environment use the Sorter transformation to sort the source port that you want to map to the FileName port of the Target transformation. After you sort the source port, map the port of the Sorter transformation to the FileName port of the Target transformation. The Data Integration Service creates only one file for each value with the same name. If you do not use the Sorter transformation, the Data Integration Service creates multiple files for each value with the same name.

  For example, Create a mapping in the Native environment or on the Spark engine to read or write an Avro file using the FileName port.

  The following image shows the sorter transformation mapping:



If you want to Map the following source port name to the FileName port of the Target transformation and write the data to an Avro target file `target1`:

Name	ID	SSN
Anna	1	1
John	4	4
Smith	4	4

Name	ID	SSN
John	5	5
Anna	2	2

Add a Sorter transformation to sort the source port and map the source port to the port of the Sorter transformation. Then, map the port of the Sorter transformation to the FileName port of the Target transformation. The Data Integration Service creates the following directories and single file per thread within the directories:

```
target1.avro=Anna
```

In this directory, the Data Integration Service creates a file with the following values: `1,1,1,2,2,2`.

```
target1.avro=John
```

In this directory, the Data Integration Service creates a file with the following values: `4,4,4,5,5,5`.

```
target1.avro=Smith
```

In this directory, the Data Integration Service creates a file with the following values: `4,4,4`.

If you do not add a Sorter transformation, the Data Integration Service creates the following directories and multiple files within the directories:

```
target1.avro=Anna
```

In this directory, the Data Integration Service creates two part files with the following values: `1,1,1` and `2,2,2`.

```
target1.avro=John
```

In this directory, the Data Integration Service creates two files with the following values: `4,4,4` and `5,5,5`.

```
target1.avro=Smith
```

In this directory, the Data Integration Service creates one file with the following values: `4,4,4`.

Flat File Data Objects

You can read data from and write data to HDFS through a fixed-width or delimited flat file data object that does not contain binary data.

You can create or import a flat file data object. The data object properties that you specify in the Developer tool must match the properties of the source file.

After you create a flat file data object, you can edit the following file properties:

- HDFS connection properties
- Compression formats

To read large volumes of data, you can connect a flat file source to read data from a directory of flat files.

You can use the flat file data objects as a source, target, or lookup transformation in mappings and mapplets. You can select the mapping environment and run the mappings in a native or Hadoop run-time environment. You can create and run profiles against flat file data objects.

When you configure a mapping that contains flat file data objects to run in the native environment, you can enable the mapping for partitioning. The Data Integration Service can use multiple partitions to read data from flat file sources with an HDFS connection. The Data Integration Service can also use multiple partitions

to write data to flat file targets with an HDFS connection. When the Data Integration Service adds partitions, it increases the number of processing threads, which can increase mapping performance.

Compression and Decompression for Flat File Sources and Targets

File compression can increase data transfer rates and reduce space for data storage.

You can read and write compressed flat files, specify compression formats, and decompress files. You can compress and decompress files in compression formats such as Bzip2 and Lzo, or specify a custom compression format.

You can specify a file or a directory of files. When the Data Integration Service reads from a directory, it reads the files of the specified format only and ignores files of other formats.

For information about how Hadoop processes compressed and uncompressed files, see the Hadoop documentation.

The following table describes the compression options:

Compression Options	Description
None	The file is not compressed.
Auto	The Data Integration Service detects the compression format of the file based on the file extension.
Gzip	The GNU zip compression format that uses the DEFLATE algorithm.
Bzip2	The Bzip2 compression format that uses the Burrows–Wheeler algorithm.
Lzo	The Lzo compression format that uses the Lempel-Ziv-Oberhumer algorithm. **Note:** The JAR files for LZO compression are not available with the default Hadoop installation. You must place the JAR files for the LZO compression format in the `lib` folder of the distribution directory and verify the distribution directory properties.
Custom	Custom compression format. If you select this option, you must specify the fully qualified class name implementing the Hadoop `CompressionCodec` interface in the **Compression Codec** field.

Related Topics:

- "Custom Formats Configuration" on page 24

Rules and Guidelines for Flat File Data Objects

Use the following rules and guidelines when you use flat file sources with an HDFS connection:

- You cannot use a command to generate or transform flat file data and send the output to the flat file source at run time.
- You cannot use an indirect source type.

Use the following rules and guidelines when you use flat file targets with an HDFS connection:

- You cannot append output data to target files and reject files. The Data Integration Service truncates the target files and reject files before writing the data.

- You cannot use the command output type.
- When the flat file target is in a partitioned mapping, you cannot write to a merge file that contains the target output for all partitions. The Data Integration Service concurrently writes the target output to a separate file for each partition.

Configuring a Flat File Data Object with an HDFS Connection

Configure a flat file data object with an HDFS connection to read data from or write data to HDFS.

If you create an empty flat file, the file properties must match that of the file in HDFS. If you import a flat file data object, the file must reside in your local file system.

1. Click the **Advanced** tab of the flat file data object.
2. Navigate to the runtime properties for the flat file source in the **Runtime: Read** properties or the flat file target in the **Runtime: Write** properties.
3. Configure the HDFS connection properties.
4. Optionally, you can configure the compression properties.

Related Topics:

- "Flat File Data Object Read Properties" on page 25
- "Flat File Data Object Write Properties" on page 32

Naming Convention for Flat File Targets

When you run a mapping on the Blaze engine to write data to a flat file target, the Data Integration Service creates multiple target files with the following naming convention:

```
<FileName>-P1, <FileName>-P2,....<FileName>-P100....<FileName>-PN
```

The naming convention helps you to delete multiple split files generated from the previous mapping runs and to avoid the deletion of target files generated from another mapping with similar file names.

Complex File Data Objects

A complex file data object is a representation of a file in the Hadoop file system. Create a complex file data object to read or write structured, semi-structured, and unstructured data to HDFS or View File System (ViewFS).

You can read files from the local system or HDFS or ViewFS. Similarly, you can write files to the local system or HDFS or ViewFS. To read large volumes of data, you can connect a complex file source to read data from a directory of files that have the same format and properties. You can read and write compressed binary files.

When you read or write the industry-standard file formats, you may or may not use the Data Processor transformation based on the structure of the file and the engine you select to run the mapping.

You can use a complex file data object with an intelligent structure model to read and parse semi-structured or structured data from text, CSV, XML, or JSON files, as well as PDF forms, Microsoft Word tables, or Microsoft Excel. The output of the complex file data object is primitive and complex elements. You do not need to use a Data Processor transformation with a complex file data object that uses an intelligent structure model. The Data Integration Service can directly read intelligent structure model resources to HDFS or ViewFS or the local file system.

When you use a binary complex file data object as a source, you can use a Data Processor transformation to parse the file. The output of the binary complex file data object is a binary stream. Similarly, when you write binary data to a complex file, you must use a Data Processor transformation to convert the source data into a binary format. You can then use the binary stream to write data to the binary complex file.

When you create a complex file data object, a read and write operation is created. You can use the complex file data object read operation as a source in mappings and mapplets. You can use the complex file data object write operation as a target in mappings and mapplets. You can select the mapping environment and run the mappings in a native or Hadoop run-time environment.

Complex File Data Object Overview Properties

The Data Integration Service uses overview properties when it reads data from or writes data to a complex file.

Overview properties include general properties that apply to the complex file data object. They also include object properties that apply to the resources in the complex file data object. The Developer tool displays overview properties for complex files in the **Overview** view.

General Properties

The following table describes the general properties that you configure for complex files:

Property	Description
Name	The name of the complex file data object.
Description	The description of the complex file data object.
Access Method	The access method for the resource. - Connection. Select **Connection** to specify an HDFS connection. - File. Select **File** to browse for a file on your local system.
Connection	The name of the HDFS connection.

Objects Properties

The following table describes the objects properties that you configure for complex files:

Property	Description
Name	The name of the resource.
Type	The native data type of the resource.
Description	The description of the resource.
Access Type	Indicates that you can perform read and write operations on the complex file data object. You cannot edit this property.

Compression and Decompression for Complex File Sources and Targets

You can read and write compressed complex files, specify compression formats, and decompress files. You can use compression formats such as Bzip2 and Lzo, or specify a custom compression format. The compressed files must be of the binary format.

You can compress sequence files at a record level or at a block level.

For information about how Hadoop processes compressed and uncompressed files, see the Hadoop documentation.

The following table describes the complex file compression formats for binary files:

Compression Options	Description
None	The file is not compressed.
Auto	The Data Integration Service detects the compression format of the file based on the file extension.
DEFLATE	The DEFLATE compression format that uses a combination of the LZ77 algorithm and Huffman coding.
Gzip	The GNU zip compression format that uses the DEFLATE algorithm.
Bzip2	The Bzip2 compression format that uses the Burrows–Wheeler algorithm.
Lzo	The Lzo compression format that uses the Lempel-Ziv-Oberhumer algorithm.
Snappy	The LZ77-type compression format with a fixed, byte-oriented encoding. **Note:** Default compression format is Snappy on the Spark engine.
Custom	Custom compression format. If you select this option, you must specify the fully qualified class name implementing the `CompressionCodec` interface in the **Custom Compression Codec** field.

Related Topics:

- "Custom Formats Configuration" on page 24

Parameterization of Complex File Data Objects

You can parameterize the complex file connection and the complex file data object operation properties.

You can parameterize the following data object read operation properties for complex data objects:

- Connection in the run-time properties
- File Format, Input Format, Compression Format, and Custom Compression Codec in the advanced properties.
- Schema Format and Schema in schema properties.

You can parameterize the following data object write operation properties for complex file data objects:

- Connection in the run-time properties.
- File Format, File Name, Output Format, Output Key Class, Output Value Class, Compression Format, Custom Compression Codec, and Sequence File Compression Type in the advanced properties.
- Schema Format and Schema in schema properties.

Note: When you parameterize the schema, ensure that the source file format and the data types are supported in the engine where you run the mapping.

The following attributes support full and partial parameterization for complex file data objects:

- File Path in the advanced properties of the data object read operation.
  For example, to parameterize a part of the attribute value where the file path in the advanced property is `/user/adpqa/dynschema.txt`, create a parameter as *$str="/user/adpqa"*, and then edit the file path as `$str/dynschema.txt`. You can also parameterize the value of the entire file path.
- File Directory in the advanced properties of the data object write operation.
  For example, to parameterize a part of the attribute value where the file directory in the advanced property is `/export/home/qamercury/source`, create a parameter as *$param="/export/home"*, and then edit the file directory as `$param/qamercury/source`. You can also parameterize the value of the entire directory.

Complex File Data Object Output Parsing

You can use an Avro or Parquet format complex file data object as a source or target without using a Data Processor transformation. The Data Integration Service can directly read and write Avro and Parquet resources that contain flat structure to HDFS or local file system.

You can use a complex file data object with an intelligent structure model resource as a source in a mapping that runs over Spark. When you associate a complex file data object with an intelligent structure model, you can use any file input that the intelligent structure model applies to without using a Data Processor transformation.

When you use a binary complex file data object as a source, you can use a Data Processor transformation to parse the binary output of the complex file.

Configure the Data Processor transformation as follows:

- Set an input port to buffer input and binary data type. Specify the port size. The port size that you specify in the complex file properties and the Data Processor transformation must be the same.
- Set an output port to buffer output or set it for relational output. If you set the ports for relational output, specify the ports based on the number of relational groups of ports you want in the output. Specify the port size for the ports. You can use an XML schema reference that describes the XML hierarchy.
- Set a Streamer object as a startup component.

If you configure a binary complex file data object with an intelligent structure model, you do not need to use a Data Processor transformation to parse the output of the complex file.

When you use a complex file data object as a target, you must use a Data Processor transformation to convert the source data into a binary format. Set the Data Processor transformation port to binary. You can then use the binary stream as an input to the complex file data object.

Creating a Complex File Data Object

Create a complex file data object to read data from or write data to HDFS or View File System (ViewFS).

1. Select a project or folder in the **Object Explorer** view.

2. Click **File** > **New** > **Data Object**.
3. Select **Complex File Data Object** and click **Next**.

   The **New Complex File Data Object** dialog box appears.
4. Optionally, enter a name for the data object.
5. Click **Browse** next to the **Location** option and select the target project or folder.
6. In the **Resource Format** list, select any of the following formats:
   - Intelligent Structure Model: to read any format that an intelligent structure parses.
   - Binary: to read any resource format.
   - Avro: to read an Avro resource.
   - Parquet: to read a Parquet resource.
   - JSON: to read a JSON resource.
   - Orc: to read an Orc resource.
   - XML: to read an XML resource.

   Note: Intelligent structure model is supported only in Spark mode.
7. In the **Access Type** list, select **Connection** or **File**.
   - Select **Connection** to access a file on HDFS or ViewFS. Click **Browse** next to the **Connection** option and select an HDFS or ViewFS connection. Click **Add** next to the **Selected Resource** option to add a resource to the data object. If a default Metadata Access Service is not set, a message appears to configure the Metadata Access Service. Click **OK** and set one Metadata Access Service as default. After you set a default Metadata Access Service, the **Add Resource** dialog box appears. If the Metadata Access Service does not exist, contact the Informatica administrator to create a new Metadata Access Service in the domain. Navigate or search for the resources to add to the data object and click **OK**.
   - Select **File** to access a file on your local system. Click **Browse** next to the **Resource Location** option and select the file that you want to add. Click **Fetch**. The selected file is added to the **Selected Resources** list.

   Note: To use an intelligent structure model, for the **Selected Resource** option, browse to and select the appropriate `.amodel` file.
8. From the **Available OS Profiles** list, select an operating system profile. You can use the **Available OS Profiles** to increase security and to isolate the design-time user environment when you import and preview metadata from a Hadoop cluster.

   Note: The Developer tool displays the **Available OS Profiles** list only if the Metadata Access Service is enabled to use operating system profiles. The Metadata Access Service imports the metadata with the default operating system profile assigned to the user. You can change the operating system profile from the list of available operating system profiles.
9. Click **Finish**.

   The data object appears under the Physical Data Objects category in the project or folder in the **Object Explorer** view. A read and write operation is created for the data object. Depending on whether you want to use the complex file data object as a source or target, you can edit the read or write operation properties. You can also create multiple read and write operations for a complex file data object. For a data object with an intelligent structure model, create a read operation. You cannot use a write transformation for a data object with an intelligent structure model in a mapping.

   Note: The complex file data object write operation goes through and the mapping runs successfully even if you have unconnected ports for required fields in the Parquet resource type. The NULL values are

inserted in the target object when such a mapping runs. The complex file data object read operation results in an error while reading NULL values from the Parquet resource as Parquet Example Object Model does not support NULL read.

10. For a read operation with an intelligent structure model, specify the path to the input file. In the **Data Object Operations** panel, select the **Advanced** tab. In the **File path** field, specify the path to the input file.

Creating a Complex File Object Read or Write Operation

You can add an complex file data object read or write operation to a mapping or mapplet as a source.

Before you create an complex file data object read or write operation, you must create at least one complex file data object. You can create the data object read or write operation for one or more complex file data objects.

Perform the following steps to create an complex file data object read or write operation:

1. Select the data object in the **Object Explorer** view.
2. Right-click and select **New** > **Data Object Operation**.

   The **Data Object Operation** dialog box appears.
3. Enter a name for the data object read or write operation.
4. Select **Read** or **Write** as the type of data object operation.
5. Click **Add**.

   The **Select Resources** dialog box appears.
6. Select the complex file object for which you want to create the data object read or write operation and click **OK**.
7. Click **Finish**.

The Developer tool creates the data object read or write operation for the selected data object.

Rules and Guidelines for Creating a Complex File Data Object Operation

Use the following rules and guidelines when you create an complex file data object operation:

- When you create a data object read or write operation, you can add new columns or modify the columns in the **Ports** tab directly.
- To modify the columns of a complex file, you must reconfigure the column projection properties.
- When you create a mapping to read or write a JSON complex file, the Developer Tool uses the first record in the JSON file as a sample for projection. If the value of an attribute in the sample is null, The Developer Tool defaults its type to "string". You can modify the columns under **Enable Column Projection** for data object operations.
- To modify the columns of an Avro, JSON, ORC, or Parquet file, change the complex file file format in the **Schema** field of the schema properties.
- When you create a mapping to read or write an Avro, JSON, ORC, or Parquet file, you can copy the columns of the Source transformations, Target transformations, or any other transformations from the **Ports** tab. Then, you can paste the columns in the data object read or write operation directly.

- When you copy the columns from any transformation to the data object read or write operation, you can change the data type of the columns. The Data Integration Service resets the precision value of the data type to the default value.
  However, the Data Integration Service does not change the precision value of the String data type to the default value.

Custom Formats

Custom formats provide flexibility with the input, output, and compression formats that you can use with PowerExchange for HDFS.

Apart from the input, output, and compression formats that PowerExchange for HDFS supports, you can use custom formats to read, write, and compress files. You can use the custom formats that Hadoop supports.

You can specify the following custom formats:

- Custom input format for complex file data objects
- Custom output format for complex file data objects
- Custom compression format for flat file and complex file data objects

Custom Formats Configuration

Before you use custom formats, you must complete configuration tasks in the Informatica environment.

To use custom formats in the native environment, copy the .jar files that implement the custom formats to the following directory:

```
<Informatica installation directory>/services/shared/hadoop/<hadoop distribution name>/
infaLib
```

To use custom formats in the Hadoop environment, see the Hadoop documentation for information about the prerequisite tasks.

If the custom compression includes native libraries, depending on the run-time environment, add the path of the native libraries to the environment variable $LD_LIBRARY_PATH or to the Hadoop connection. If you use the native environment, add the path of the native libraries to the environment variable $LD_LIBRARY_PATH. If you use the Hadoop environment, add the path of the native libraries to the Hadoop connection.

Perform the following steps to add the path of the native libraries to the Hadoop connection:

1. Click the **Common Attributes** tab in the Hadoop connection.
2. Under the **Common Properties** section, click **Edit** next to the **Advanced Properties** field.
3. Add the infapdo.java.opts property and set its value to the path of the native libraries.
   For example, the following property specifies a native library path for a Cloudera distribution:

   ```
   infapdo.java.opts=-Djava.library.path=$HADOOP_NODE_INFA_HOME/services/shared/
   bin:$HADOOP_NODE_INFA_HOME/services/shared/hadoop/CDH_5.13/lib/native
   ```

   Note: If you use Hortonworks or MapR distributions, change the native library path based on the distribution.

CHAPTER 5

HDFS Data Extraction

This chapter includes the following topics:

- HDFS or View File System (ViewFS) Data Extraction Overview, 25
- Flat File Data Object Read Properties, 25
- Complex Files Partitioning, 26
- Complex File Data Object Read Properties, 26

HDFS or View File System (ViewFS) Data Extraction Overview

You can use a flat file data object or a complex file data object to read data from HDFS or View File System (ViewFS).

Complete the following tasks to read data from HDFS or ViewFS by using PowerExchange for HDFS:

1. Create an HDFS or ViewFS connection.
2. Create a flat file data object or a complex file data object. Specify the data object properties such as the file location, compression format, and input format.
3. Create a mapping and use the flat file data object or the complex file data object read operation as a source.
4. If needed, configure a Data Processor transformation to parse the complex file.
5. Configure the validation and run-time environment type.
6. Run the mapping to read data from HDFS or ViewFS.

Flat File Data Object Read Properties

The Data Integration Service uses read properties when it reads data from a flat file. You can edit the format and runtime read properties on the **Advanced** tab.

The following table describes the HDFS or View File System (ViewFS) connection and compression run-time properties that you configure for flat file sources:

Property	Description
Connection Type	The type of connection. Select from the following options: - None. The source file does not require a connection. - Hadoop File System. The source file resides in HDFS. - View File System (ViewFS). The source file resides in ViewFS. Default is None.
Connection Name	The name of the connection. Select an HDFS or ViewFS connection or assign a mapping parameter that defines the connection details.
Compression Format	Optional. Specifies the compression format. Select from the following options: - None - Auto - Gzip - Bzip2 - Lzo - Custom
Compression Codec	Required for custom compression. Specify the fully qualified class name implementing the Hadoop `CompressionCodec` interface.

Complex Files Partitioning

When you run a mapping in a Hadoop environment to read data from sequence files and custom input format files that are splittable, the Data Integration Service uses multiple partitions to read data from the source. The Data Integration Service creates multiple Map jobs to read data in parallel, thereby resulting in high performance.

To read text files in parallel, specify the following input format in the complex file read properties:

```
com.informatica.adapter.hdfs.hadoop.io.InfaTextInputFormat
```

You can also specify the following input format to read text files in batches:

```
com.informatica.adapter.hdfs.hadoop.io.InfaBatchTextInputFormat
```

Typically, when you read complex files, the Data Processor transformation has a Streamer component and a Parser component. By default, the Data Integration Service calls the Data Transformation Engine for every record. You can modify this behavior by using the count property in the Streamer component. Set the count property to define the number of records that the Data Integration Service must treat as a batch. When you set the count property, the Data Integration Service calls the Data Transformation Engine for each batch of records instead of calling the Data Transformation Engine for every record. Since the Data Integration Service processes the text files in batches, the performance increases.

Complex File Data Object Read Properties

The Data Integration Service uses read properties when it reads data from a complex file. Select the Output transformation to edit the general, ports, sources, and run-time properties.

Note: The FileName port is displayed by default when you create a data object read operation. You can remove the FileName port if you do not want to read the FileName data.

Wildcard Characters for Reading Data from Complex Files

When you run a mapping in the native environment or on the Spark engine to read data from complex files, you can use wildcard characters to specify the source directory name or the source file name. You can use wildcard characters to specify the absolute path or relative path.

To use wildcard characters for the source directory name or the source file name, select the **Allow Wildcard Characters** option in the advanced read properties of the complex file data object. You can then use wildcard characters in the **File path** field.

You can use the following wildcard characters:

? (Question mark)

The question mark character (?) allows one occurrence of any character. For example, if you enter the source file name as `a?b.txt`, the Data Integration Service reads data from files with the following names:

- `a1b.txt`
- `a2b.txt`
- `aab.txt`
- `acb.txt`

*** (Asterisk)**

The asterisk mark character (*) allows zero or more than one occurrence of any character. If you enter the source file name as `a*b.txt`, the Data Integration Service reads data from files with the following names:

- `aab.txt`
- `a1b.txt`
- `ab.txt`
- `abc11b.txt`

Combination of * (Asterisk) and ? (Question mark)

The combination of asterisk mark character (*) and question mark character (?) allows zero or more than one occurrence of any character.

General Properties

The Developer tool displays general properties for complex file sources in the **Read** view.

The following table describes the general properties that you configure for complex file sources:

Property	Description
Name	The name of the complex file. This property is read-only. You can edit the name in the **Overview** view. When you use the complex file as a source in a mapping, you can edit the name in the mapping.
Description	The description of the complex file.

Ports Properties

Ports properties for a physical data object include port names and port attributes such as data type and precision.

Note: The port size specified in the source transformation and Output transformation must be the same.

The following table describes the ports properties that you configure for complex file sources:

Property	Description
Name	The name of the resource.
Type	The native data type of the resource.
Precision	The maximum number of significant digits for numeric data types, or the maximum number of characters for string data types.
Description	The description of the resource.

Schema Properties

The Developer tool displays the schema properties for intelligent structure model, Avro, JSON, ORC and Parquet complex file sources in the Properties view of the **Read** operation.

The following table describes the Schema properties that you configure for the complex file sources:

Property	Description
Column Name	Displays the name of the column.
Column Type	Displays the format of the column.
Enable Column Projection	Displays the column details of the complex files sources.
Schema Format	Displays the schema format that you selected while creating the complex file data object. You can change the schema format and provide respective schema. You can select one of the following options: - Avro - Json - Orc - Parquet - Xml - Intelligent Structure Model - Assign Parameter You can change the complex file format without losing the column metadata even after you configure the column projection properties for another complex file format. You can parameterize the schema format using the **Assign Parameter** option. **Note:** You can switch from one schema format to another only once. If you change the schema format more than once, you might lose the original datatypes.

Property	Description
Schema	Displays the schema associated with the complex file. You can select a different schema. You can select one of the following options: - Browse - Assign Parameter - Assign Path as Parameter For the Assign Path as Parameter option, the path can be obtained from the server. When you use Refresh Schema for the source or target in a mapping and also, parameterize the schema, the parameterized schema takes precedence over the refresh schema. **Note:** - If you disable the column projection, the schema associated with the complex file is removed. If you want to associate schema again with the complex file, enable the column projection and click Select Schema. - When you parameterize the schema in a Parquet complex file, the schema should not contain a String data type, use UTF8 data type instead.
Column Mapping	Displays the mapping between input and output ports. **Note:** If you disable the column projection, the mapping between input and output ports is removed. If you want to map the input and output ports, enable the column projection and click Select Schema to associate a schema to the complex file.

Note: In the native environment, Data Preview and a mapping may fail in the following scenarios:

- When you import an Avro file as a source object and switch the schema format to Parquet and select a Parquet file as the source object.
- When you import a JSON file as a source object and switch the schema format to Avro and select an Avro file as the source object.

You must edit the schema as per the selected schema format or enable the refresh schema at runtime option in the mapping if you want to change the schema format.

Sources Properties

The Developer tool displays the sources properties for complex file sources in the Output transformation in the **Read** view.

The sources properties list the resources of the complex file data object. You can add or remove resources in the data object.

Advanced Properties

The Developer tool displays the advanced properties for complex file sources in the Output transformation in the **Read** view.

The following table describes the advanced properties that you configure for complex file sources:

Property	Description
Allow Wildcard Characters	Indicates whether you want to use wildcard characters for the source directory name or the source file name. If you select this option, you can use wildcard characters ? and * for the source directory name or the source file name in the **File path** field. The question mark character (?) allows one occurrence of any character. The asterisk character (*) allows zero or more than one occurrence of any character. This option is applicable when you run a mapping in the native environment or on the Spark engine.
File Format	The file format. Select one of the following file formats: - Binary. Select Binary to read any file format. - Sequence. Select Sequence File Format for source files of a Hadoop-specific binary format that contain key and value pairs. - Custom Input. Select Input File Format to specify a custom input format. You must specify the class name implementing the `InputFormat` interface in the **Input Format** field. - Assign Parameter. Select Assign Parameter to parameterize the file format. Default is Binary.

Input Format	The class name for files of the input file format. If you select **Input File Format** in the **File Format** field, you must specify the fully qualified class name implementing the `InputFormat` interface.
	To read files that use the Avro format, use the following input format:
	`com.informatica.avro.AvroToXML`
	To read files that use the Parquet format, use the following input format:
	`com.informatica.parquet.ParquetToXML`
	You can use any class derived from `org.apache.hadoop.mapreduce.InputFormat`.

Input Format Parameters	Parameters for the input format class. Enter name-value pairs separated with a semicolon. Enclose the parameter name and value within double quotes. For example, use the following syntax: `"param1"="value1";"param2"="value2"`
Compression Format	Optional. The compression format for binary files. Select one of the following options: - None - Auto - DEFLATE - gzip - bzip2 - Lzo - Snappy - Custom

Property	Description
Custom Compression Codec	Required for custom compression. Specify the fully qualified class name implementing the `CompressionCodec` interface.
File path	The location of the file or directory. If the path is a directory, all the files in the directory must have the same file format. If the file or directory is in HDFS, enter the path without the node URI. For example, `/user/lib/testdir` specifies the location of a directory in HDFS. The path must not contain more than 512 characters. If the file or directory is in the local system, enter the fully qualified path. For example, `/user/testdir` specifies the location of a directory in the local system. **Note:** The Data Integration Service ignores any subdirectories and their contents. If you select the **Allow Wildcard Characters** option, you can use wildcard characters ? and * for the source directory name or the source file name.

CHAPTER 6

HDFS Data Load

This chapter includes the following topics:

- HDFS or View File System (ViewFS) Data Load Overview, 32
- Flat File Data Object Write Properties, 32
- Complex File Streaming, 33
- Complex Files Output Collection Mode, 34
- Complex File Data Object Write Properties, 35

HDFS or View File System (ViewFS) Data Load Overview

You can use a flat file data object or a complex file data object to write data to HDFS or ViewFS.

Complete the following tasks to write data to HDFS or ViewFS by using PowerExchange for HDFS:

1. Create an HDFS or ViewFS connection.
2. Create a flat file data object or a complex file data object. Specify the data object properties such as the file location and compression format.
3. Create a mapping and use the flat file data object or the complex file data object write operation as a target.
4. Configure the validation and run-time environment type.
5. Run the mapping to write data to HDFS or ViewFS.

Flat File Data Object Write Properties

The Data Integration Service uses write properties when it writes data to a flat file. You can edit the format and runtime write properties on the **Advanced** tab.

The following table describes the HDFS or View File System (ViewFS) connection and compression properties that you configure for flat file targets:

Property	Description
Connection Type	The type of connection. Select from the following options: - None. The target file does not require a connection. The target file location is specified by the output file directory. - Hadoop File System. The target file is in HDFS. - View File System (ViewFS). The target file is in ViewFS. Default is None.
Connection Name	The name of the connection. Select an HDFS or ViewFS connection or assign a mapping parameter that defines the connection details.
Compression Format	Optional. Specifies the compression format. Select from the following options: - None - Gzip - Bzip2 - Lzo - Custom
Compression Codec	Required for custom compression. Specify the fully qualified class name implementing the Hadoop `CompressionCodec` interface.

Complex File Streaming

To write data to a complex file, include a Data Processor transformation in the mapping to convert the source data into a binary format. You can use the binary stream to write data to the complex file.

The Data Processor transformation continually streams and sends input to the complex file target. It sends end of file information after it fully streams a file. It sends end of streaming information when it streams the entire input fully.

When the Data Processor transformation sends portions of the input to the complex file target, PowerExchange for HDFS appends unique identifier information to the file name. The Data Integration Service uses the unique identifiers to recognize that the streaming is in progress and not complete. Therefore, the file name that you specify in the complex file write properties is not the same as the output file in HDFS or View File System (ViewFS). The output file name in HDFS or ViewFS contains the unique identifier information as well.

The unique identifier format depends on whether the file is not compressed or not. The following table describes the unique identifier format based on whether the file is compressed or not:

Run-time Environment Type	File Type	Unique Identifier Format
Native	Uncompressed File	<filename>_<unique identifier>_<seq>.<ext>
Native	Compressed File	<filename>_<unique identifier>_<seq>.<compression format extension>

If you do not include the compression format extension as part of the file name in the complex file write properties, PowerExchange for HDFS appends extensions based on the compression format.

The following table describes the extensions that PowerExchange for HDFS appends based on the compression format that you use:

Compression Format	File Name Extension that PowerExchange for HDFS Appends
DEFLATE	.deflate
Gzip	.gz
Bzip2	.bz2
Lzo	.lzo
Snappy	.snz

Complex Files Output Collection Mode

When you write data to complex files, you can choose to collect the input rows and write the output to a single file, or create an output row for each input row.

You can specify the output collection mode in the Data Processor transformation based on the complex file type.

To specify the output collection mode in the Data Processor transformation, open the Data Processor transformation and click the **Settings** view. In the **Binary output collection mode** section, specify the output collection mode.

The following table describes the options that you can select for the output collection mode:

Property Name	Property Description
Collect input rows to a single output	Select this option if you want to collect all input rows and write the output to a single file.
Split output when size exceeds	When you write the output to a single file, you can choose to split the output file when it exceeds a particular size. Enter the size in MB exceeding which the file must be split. Default is 100 MB.
Output row for each input row (do not collect)	Select this option if you want to write an output row for each input row.

Output Collection Mode for Binary Files

When you write to binary files in a native or Hadoop environment, you can specify the output collection mode in the Data Processor transformation.

Output Collection Mode for Sequence Files and Custom Output Format Files

When you write to sequence files or custom output format files in a native environment, PowerExchange for HDFS writes all the key-value pairs into one output file. The number of key-value pairs that PowerExchange

for HDFS writes depends on the output collection mode that you specified in the Data Processor transformation.

Complex File Data Object Write Properties

The Data Integration Service uses write properties when it writes data to a complex file. Select the Input transformation to edit the general, ports, sources, and advanced properties.
Note: Though the FileName port is displayed by default when you create a data object write operation, the FileName port is not supported for the data object write operation.

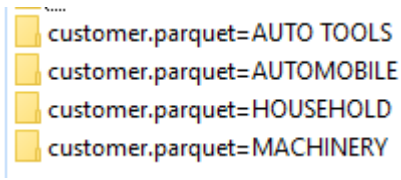
Overwriting Complex File Targets

When you run a mapping in the native environment or on the Spark engine to write data to a complex file target, you can choose to overwrite the target data. You can select the **Overwrite Target** option in the advanced write properties of the complex file data object.

If you select the **Overwrite Target** option, the Data Integration Service deletes the target data before writing data. If you do not select this option, the Data Integration Service creates a new file in the target and writes the data to the file.

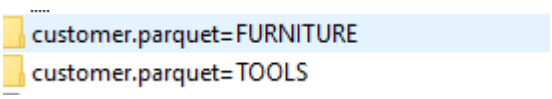
If you select the **Overwrite Target** option, the Data Integration Service deletes all the files and folders in the target directory that are prefixed with the target file name.

For example, the following image shows target data with Overwrite Fileport connected:



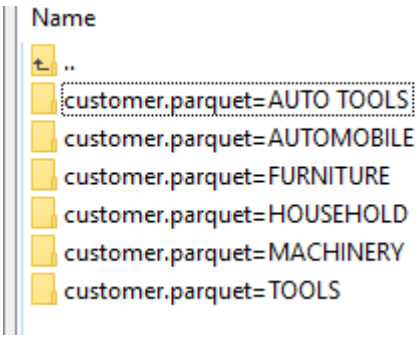
If you select the **Overwrite Target** option, the Data Integration Service deletes all the folders and files in the target directory that are prefixed with the target file name.

The following image shows the file structure after using the **Overwrite Target** option:



If you do not use the **Overwrite Target** option, the Data Integration Service adds the data to the existing files and folders in the target directory.

The following image shows the file structure when you append data:



To avoid unnecessary deletion of files and folders, you must verify that the target directory does not contain any folder or file with the same name as the target file name.

General Properties

The Developer tool displays general properties for complex file targets in the **Write** view.

The following table describes the general properties that you configure for complex file targets:

Property	Description
Name	The name of the complex file. This property is read-only. You can edit the name in the **Overview** view. When you use the complex file as a target in a mapping, you can edit the name in the mapping.
Description	The description of the complex file.

Port Properties

Port properties for a physical data object include port names and port attributes such as data type and precision.

Note: The port size specified in the target transformation and Input transformation must be the same.

The following table describes the ports properties that you configure for complex file targets:

Property	Description
Name	The name of the resource.
Type	The native data type of the resource.
Precision	The maximum number of significant digits for numeric data types, or the maximum number of characters for string data types.
Description	The description of the resource.

Schema Properties

The Developer tool displays the schema properties for Avro, JSON, ORC and Parquet complex file targets in the Properties view of the **Write** operation.

The following table describes the schema properties that you configure for Avro, JSON, and Parquet complex file targets:

Property	Description
Column Name	Displays the name of the column.
Column Type	Displays the format of the column.
Enable Column Projection	Displays the column details of the complex file targets.

Property	Description
Schema Format	Displays the schema format that you selected while creating the complex file data object. You can change the schema format and provide respective schema. You can select one of the following options: - Avro - Json - Parquet - Orc - Xml - Assign Parameter You can change the complex file format without losing the column metadata even after you configure the column projection properties for another complex file format. **Note:** You can switch from one schema format to another only once. If you change the schema format more than once, you might lose the original datatypes.
Schema	Displays the schema associated with the complex file. You can select a different schema. You can select one of the following options: - Browse - Assign Parameter - Assign Path as Parameter When you use Refresh Schema for the source or target in a mapping and also, parameterize the schema, the parameterized schema takes precedence over the refresh schema. **Note:** - If you disable the column projection, the schema associated with the complex file is removed. If you want to associate schema again with the complex file, enable the column projection and click Select Schema. - When you parameterize the schema in a Parquet complex file, the schema should not contain a String data type, use UTF8 data type instead.
Column Mapping	Displays the mapping between input and output ports. **Note:** If you disable the column projection, the mapping between input and output ports is removed. If you want to map the input and output ports, enable the column projection and click Select Schema to associate a schema to the complex file.

Target Properties

The Developer tool displays the Target properties for complex file targets in the output transformation in the **Write** view.

The target properties list the resources of the complex file data object. You can add or remove resources in the data object.

Advanced Properties

The Developer tool displays the advanced properties for complex file targets in the Input transformation in the **Write** view.

The following table describes the advanced properties that you configure for complex file targets:

Property	Description
File Directory	The directory location of the complex file target. If the directory is in HDFS, enter the path without the node URI. For example, `/user/lib/testdir` specifies the location of a directory in HDFS. The path must not contain more than 512 characters. If the directory is in the local system, enter the fully qualified path. For example, `/user/testdir` specifies the location of a directory in the local system. **Note:** The Data Integration Service ignores any subdirectories and their contents.
File Name	The name of the output file. PowerExchange for HDFS appends the file name with a unique identifier before it writes the file to HDFS. In spark mode PowerExchange for HDFS appends the file name with .avro extension.
Overwrite Target	Indicates whether the Data Integration Service must first delete the target data before writing data. If you select the **Overwrite Target** option, the Data Integration Service deletes the target data before writing data. If you do not select this option, the Data Integration Service creates a new file in the target and writes the data to the file. This option is applicable when you run a mapping in the native environment or on the Spark engine to write data to complex files.
File Format	The file format. Select one of the following file formats: - Binary. Select Binary to write any file format. - Sequence. Select Sequence File Format for target files of a Hadoop-specific binary format that contain key and value pairs. - Custom Output. Select Output Format to specify a custom output format. You must specify the class name implementing the `OutputFormat` interface in the **Output Format** field. - Assign Parameter. Select Assign Parameter to parameterize the file format. Default is Binary.
Output Format	The class name for files of the output format. If you select Output Format in the **File Format** field, you must specify the fully qualified class name implementing the `OutputFormat` interface.
Output Key Class	The class name for the output key. If you select Output Format in the **File Format** field, you must specify the fully qualified class name for the output key. You can specify one of the following output key classes: - BytesWritable - Text - LongWritable - IntWritable **Note:** PowerExchange for HDFS generates the key in ascending order.
Output Value Class	The class name for the output value. If you select Output Format in the **File Format** field, you must specify the fully qualified class name for the output value. You can use any custom writable class that Hadoop supports. Determine the output value class based on the type of data that you want to write. **Note:** When you use custom output formats, the value part of the data that is streamed to the complex file data object write operation must be in a serialized form.

Property	Description
Compression Format	Optional. The compression format for binary files. Select one of the following options: - None - Auto - DEFLATE - gzip - bzip2 - LZO - Snappy - Custom - Assign Parameter...
Custom Compression Codec	Required for custom compression. Specify the fully qualified class name implementing the `CompressionCodec` interface.
Sequence File Compression Type	Optional. The compression format for sequence files. Select one of the following options: - None - Record - Block - Assign Parameter...
Partition Option	The option to specify the partitioned columns. Select one of the following options: - None - Last N Columns Partitioned - Partition Column Names - Assign Parameter... **Note:** Applicable to dynamic mappings only.
Partition Arguments	Applicable when you select the partition option. If the partition option is **Last N Columns**, enter the number to pick the last n partition columns. If the partition option is column names, enter comma separated values. **Note:** Applicable to dynamic mappings only.

CHAPTER 7

HDFS Mappings

This chapter includes the following topics:

- HDFS or View File System (ViewFS) Mappings Overview, 40
- Complex Files Target Creation, 41
- Directory-Level Partitioning, 41
- Creating a Complex File Target from an Existing Transformation , 45
- Mapping Validation and Run-time Environments, 46
- Audits, 46
- Rules and Guidelines for Complex File Sources and Targets in a Mapping, 46
- HDFS or View File System (ViewFS) Data Extraction Mapping Example, 47
- HDFS or View File System (ViewFS) Data Load Mapping Example, 48
- HDFS or View File System (ViewFS) Avro Read Mapping Example, 50

HDFS or View File System (ViewFS) Mappings Overview

After you create a flat file or a complex file data object operation, you can create an HDFS or ViewFS mapping.

You can define the following objects in an HDFS or ViewFS mapping:

- A flat file data object or a complex file data object read operation as the input to read data from HDFS or ViewFS
- Transformations
- A flat file data object or a complex file data object write operation as the output to write data to HDFS or ViewFS

If you use a complex file data object as a source, you must use a Data Processor transformation to parse the file. Similarly, when you use a complex file data object as a target, you must use a Data Processor transformation to convert the source data into a binary format. You can then use the binary stream to write data to the complex file.

You can use complex file sources and targets as dynamic sources and targets in a mapping. For information about dynamic mappings, see the *Informatica Developer Mapping Guide*.

Validate and run the mapping. You can deploy the mapping and run it or add the mapping to a Mapping task in a workflow and run the workflow. You can also run the mapping in a Hadoop run-time environment.

Complex Files Target Creation

You can create a complex file target from an existing transformation in the mapping. The Developer tool can create an Avro, Parquet, ORC, or JSON complex file target.

You cannot use the **Create Target** option to create a complex file target from an existing transformation in the following scenarios:

- The existing transformation contains a port named FileName.
- The existing transformation points to a Hive table that contains a complex data type with hierarchical data.
- The existing transformation points to an Avro complex file that contains field names with special characters.

Directory-Level Partitioning

When you run a mapping on the Spark engine, you can read data from and write data to Avro, ORC, and Parquet files that are partitioned based on directories.

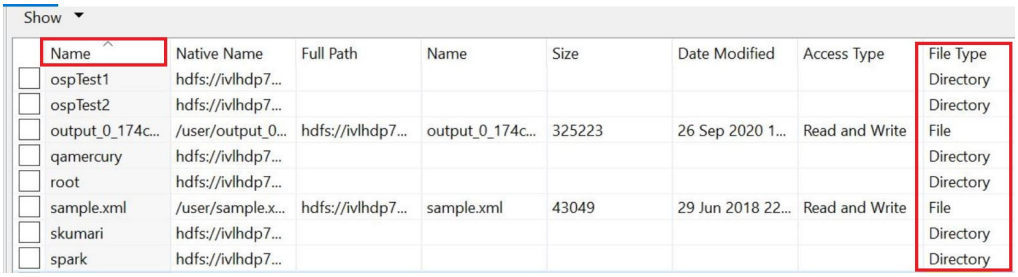
Importing a data object with partition files

Perform the following steps to import a data object to read or write from partition files:

1. Select a project or folder in the **Object Explorer** view.

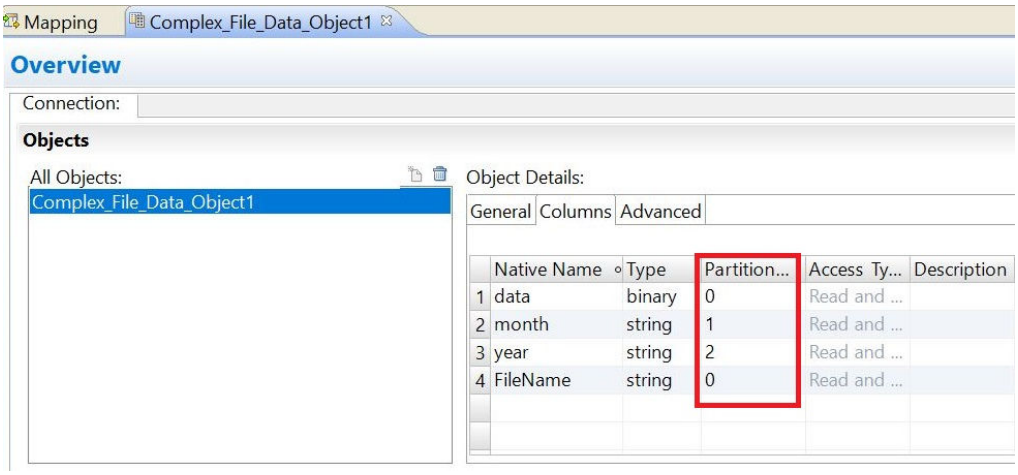
1. Click **File** > **New** > **Data Object**.
2. Select **Complex File Data Object** and click **Next**.
   The **New Complex File Data Object** dialog box appears.
3. Click **Browse** next to the **Location** option and select the target project or folder.
4. In the **Resource Format** list, select Avro, CSV, Parquet, or ORC from the menu.
5. Select **File** to access a file on your local system.
6. Click **Browse** next to the **Resource Location** option and select the file that you want to add.
7. Click **Fetch**.
   The selected file is added to the **Selected Resources** list.
8. Click **Add** next to the **Selected Resource** option to add a resource to the data object. The **Add Resource** dialog box appears.
   You can use the **File Type** column to distinguish between a directory and a file.

   The following image shows the Add resource dialogue box where you can select the file name and directory:



9. Select the file name and directory, and then Click **OK**.
10. Click **Finish**.
    The partitioned columns are displayed with the order of partitioning in the data object **Overview** tab.

    The following image shows the data object overview tab:

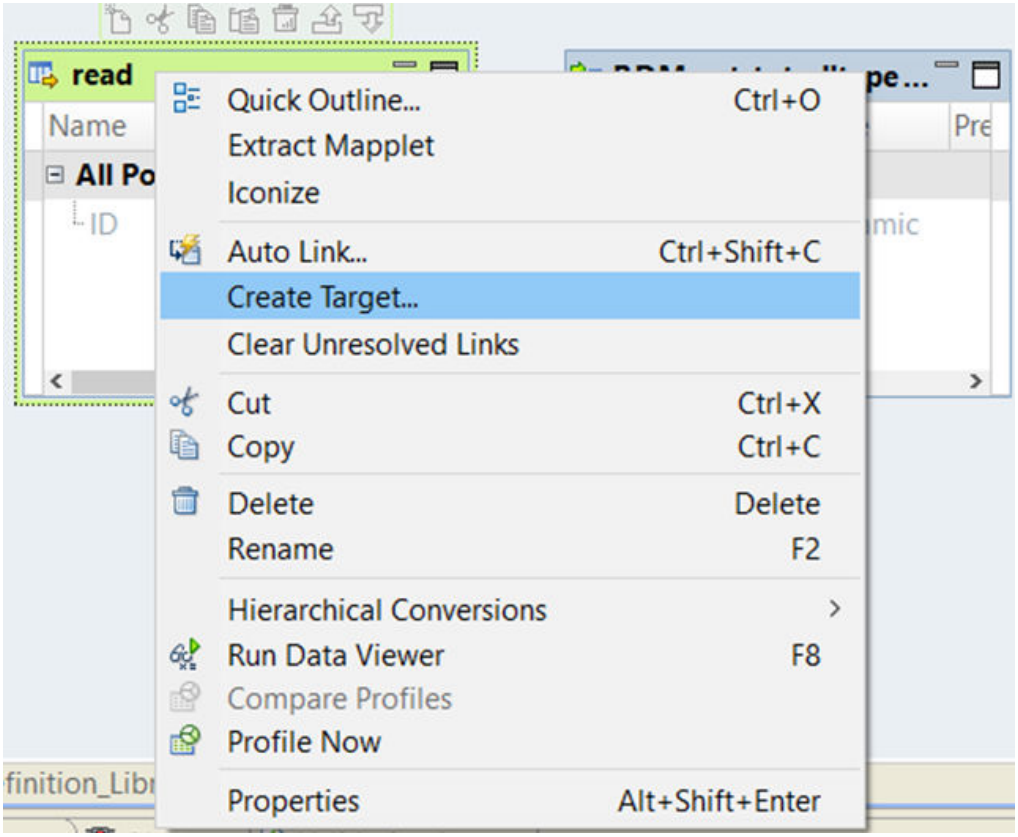


Create a target with partition files

Perform the following steps to create a target with partition files:

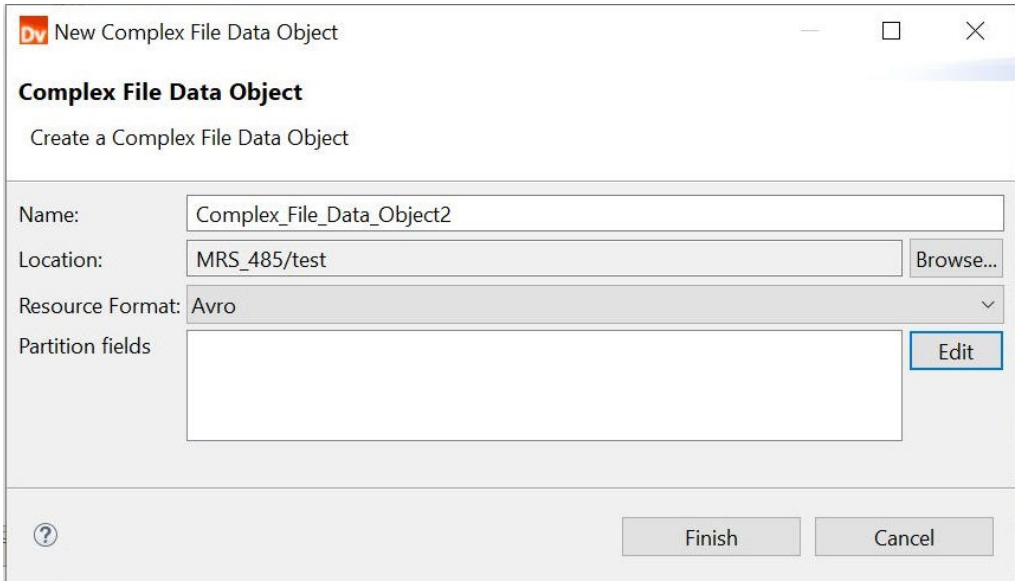
1. Select a project or folder in the **Object Explorer** view.
2. Select a source or a transformation in the mapping.
3. Right-click the transformation and select **Create Target**.
   The **Create Target** dialog box appears.

The following image shows the **Create Target** option:



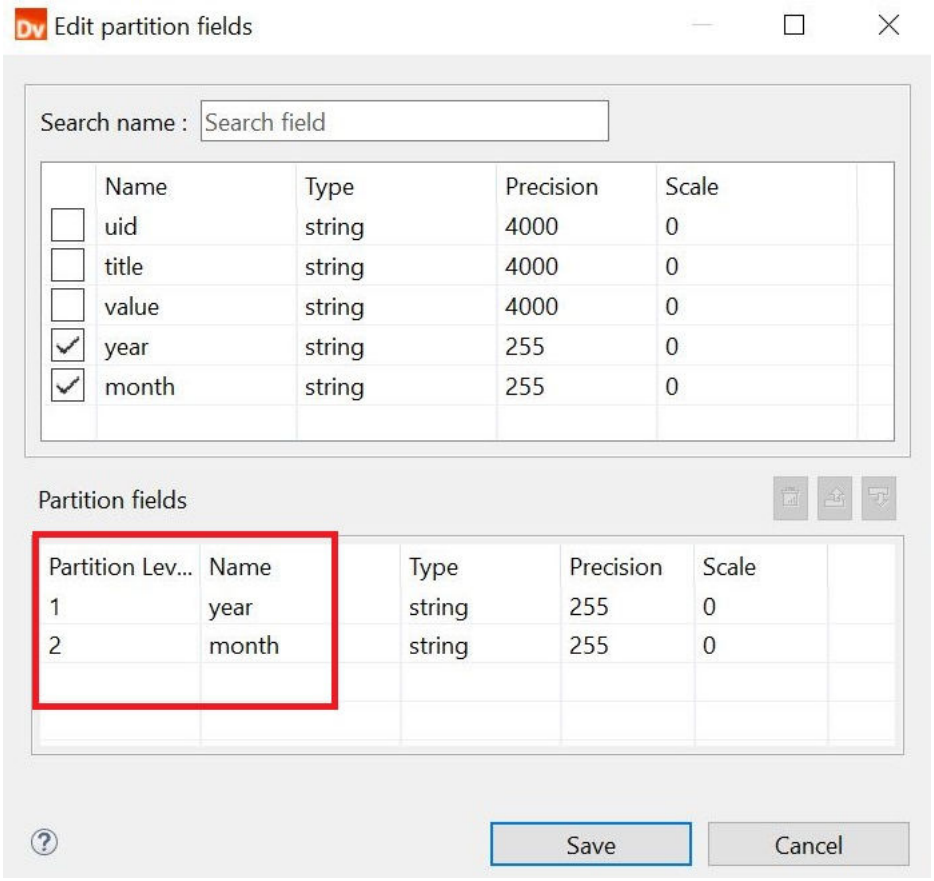
4. Select **Others** and then select **Complex File** data object from the list in the **Data Object Type** section.
5. Click **OK**.
   The **New Complex File Data Object** dialog box appears.

   The following image shows the **New Complex File Data Object** dialog box:



6. Enter a name for the data object.

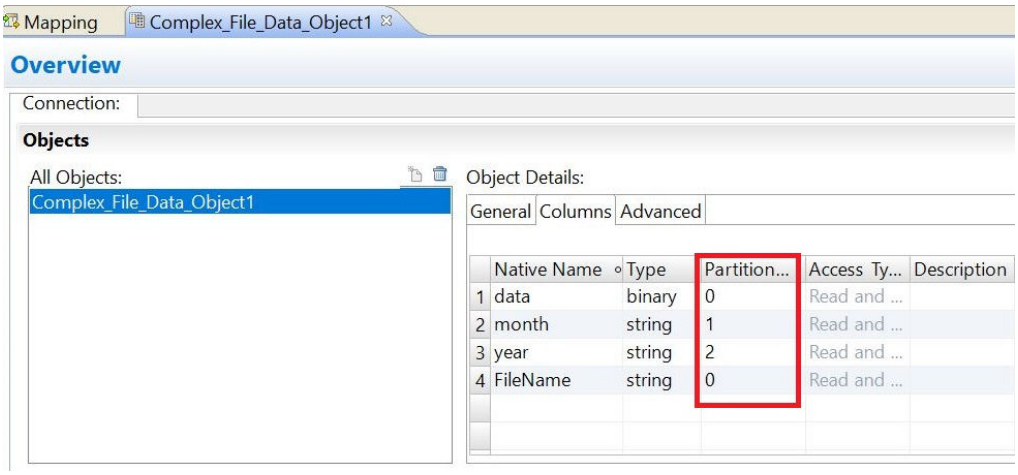
7. Enter the partition fields.
The following image shows the **Edit partition fields** dialog box:



You can change the partition order using the up and down arrows.

8. Click **Finish**.
The partitioned columns are displayed with the order of partitioning in the data object **Overview** tab.

The following image shows the data object overview tab:



Rules and Guidelines for Reading from or Writing to Partition Columns

Consider the following rules and guidelines when you read from and write to a partition folder:

- If the source contains htypes and the partitioned column includes String data types, a validation error is encountered.
- When you import a directory that has a partition folder, the data type for the partition column is imported as a String.
- For **Create Target**, you can add partition fields and arrange the partition columns in an order.
- When you import a data object, the data and FileName port always shows 0 as the partition order.
- You can read data from or write data to partition folders with Avro, Parquet, and Orc files.
- You can use only primitive data types such as String, Integer, Long, Date, and Timestamp in a partitioned column. If you specify other data types, a validation error is encountered.
- The partitioned directory that you select cannot have a partitioned column named FileName. The name is case insensitive.
- For **Create Target**, you need to remove FileName port from the target side manually, otherwise the selected files in the directory are not partitioned.

Creating a Complex File Target from an Existing Transformation

If one of the ports in the existing transformation contains a complex port, you must create a complex file target and link ports by name or link ports at run time based on the link policy.

1. Open a complex file mapping in the editor.
2. Right-click a transformation in the mapping editor and select **Create Target**.

   The **Create Target** window opens.
3. Choose the complex file data object type.
4. Choose a link type.

   You can choose from the following link types:

   Link ports by name

   Ports in the Write transformation correspond to those in the source and have the same names.

   Link dynamic port based on the mapping flow

   The Write transformation contains dynamic ports based on upstream objects in the mapping flow.

   Link ports at run time based on link policy

   Ports are created in the target at run time based on the link policy that you configure on the **Run Time Linking** tab of the Write transformation.

   For more information about dynamic ports and run-time link configuration, see the *Informatica Developer Mapping Guide*.
5. Name the new complex file data object.
6. Optionally, click **Browse** to select a location for the data object.

7. Select the complex file format as Avro, Parquet, Orc, or Json in the **Resource Format** list.
8. Click **Finish**.

The Developer tool performs the following tasks:

- Adds a complex file Write transformation to the mapping.
- Links ports.
- Creates a physical data object.
  You can configure the physical data object properties. For example, you must specify an HDFS or View File System (ViewFS) connection for the complex file data object.

Mapping Validation and Run-time Environments

You can use flat file and complex file data objects in a Hadoop run-time environment.

You can configure the mappings to run in native or Hadoop run-time environments. When you run a mapping in the native environment, the Data Integration Service processes the mapping. When you run a mapping in a Hadoop environment, the Data Integration Service can push mappings to a Hadoop cluster. You can run an HBase mapping on the Blaze or Spark engine.

For more information about the native and Hadoop environments, see the *Informatica Data Engineering Integration User Guide*.

Audits

To validate the consistency and accuracy of data processed in a mapping for a read operation, you can create an audit for the mapping.

An audit is composed of rules and conditions. Use a rule to compute an aggregated value for a single column of data. Use a condition to make comparisons between multiple rules or between a rule and constant values.

You can run audits with mappings that run on the Data Integration Service or the Spark engine.

For more information, see the *Data Engineering Integration 10.5 User Guide*.

Rules and Guidelines for Complex File Sources and Targets in a Mapping

Consider the following rules and guidelines for complex file sources and targets in a mapping:

- You cannot use the **Create Target** option to create a complex file target from an existing transformation in the following scenarios:
  - The existing transformation contains a port named FileName.

- The existing transformation points to a Hive table that contains a complex data type with hierarchical data.
- The existing transformation points to an Avro complex file that contains field names with special characters.

- If you select the **Overwrite Target** option, the Data Integration Service deletes all the files and folders in the target directory that are prefixed with the target file name.

  To avoid unnecessary deletion of files and folders, you must verify that the target directory does not contain any folder or file with the same name as the target file name.
- You cannot delete a port from a physical data object in a complex file mapping running on the Spark engine as the mapping results can be unpredictable. You can disconnect the port instead.
- You cannot use binary complex files in a mapping that runs on the Spark engine.
- When you run a mapping to read data from a complex file source in Parquet format, the Time and Timestamp data type is read according to the timezone of the cluster where data is generated.
- When Refresh schema, ApplyNewSchema, or ApplyNewColumn option is enabled for a mapping that runs on an Amazon EMR 5.29 cluster with Glue enabled as the Hive metastore, the mapping fails with the following error:

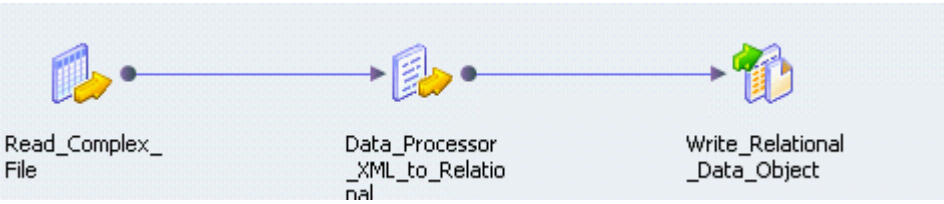
  ```
  java.lang.RuntimeException: Import of table DEFAULT.Hive_NewColumn_tar failed as table not found
  ```
- When you enable the mapping to refresh the schema and run it on a MapR cluster, the mapping fails.

HDFS or View File System (ViewFS) Data Extraction Mapping Example

Your organization needs to analyze purchase order details such as customer ID, item codes, and item quantity. The purchase order details are stored in a semi-structured compressed XML file in HDFS or ViewFS. The hierarchical data includes a purchase order parent hierarchy level and a customer contact details child hierarchy level. Create a mapping that reads all the purchase records from the file in HDFS or ViewFS. The mapping must convert the hierarchical data to relational data and write it to a relational target.

You can use the extracted data for business analytics.

The following figure shows the example mapping:



You can use the following objects in the HDFS or ViewFS mapping:

HDFS Input

The input object, Read_Complex_File, is a Read transformation that represents a compressed XML file stored in HDFS or ViewFS.

Data Processor Transformation

The Data Processor transformation, Data_Processor_XML_to_Relational, parses the XML file and provides a relational output.

Relational Output

The output object, Write_Relational_Data_Object, is a Write transformation that represents a table in an Oracle database.

When you run the mapping, the Data Integration Service reads the file in a binary stream and passes it to the Data Processor transformation. The Data Processor transformation parses the specified file and provides a relational output. The output is written to the relational target.

You can configure the mapping to run in a native or Hadoop run-time environment.

Complete the following tasks to configure the mapping:

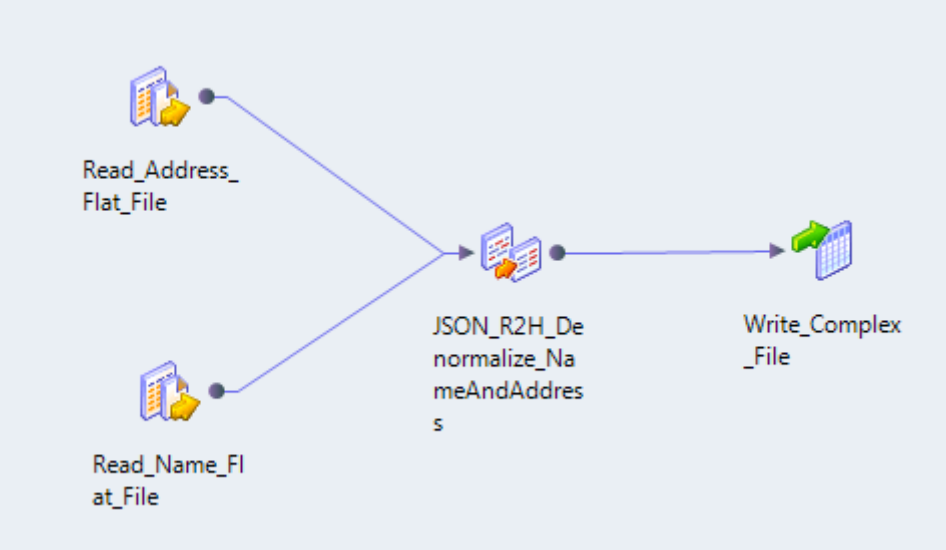
1. Create an HDFS connection to read files from the Hadoop cluster.
2. Create a complex file data object read operation. Specify the following parameters:
   - The file as the resource in the data object.
   - The file compression format.
   - The HDFS file location.
3. Optionally, you can specify the input format that the Mapper uses to read the file.
4. Drag and drop the complex file data object read operation into a mapping.
5. Create a Data Processor transformation. Configure the following properties in the Data Processor transformation:
   - An input port set to buffer input and binary data type.
   - Relational output ports depending on the number of columns you want in the relational output. Specify the port size for the ports. Use an XML schema reference that describes the XML hierarchy. Specify the normalized output that you want. For example, you can specify PurchaseOrderNumber_Key as a generated key that relates the Purchase Orders output group to a Customer Details group.
   - Create a Streamer object and specify Streamer as a startup component.
6. Create a relational connection to an Oracle database.
7. Import a relational data object.
8. Create a write transformation for the relational data object and add it to the mapping.

HDFS or View File System (ViewFS) Data Load Mapping Example

Your organization needs to denormalize employee ID, name, and address details. The employee ID, name, and address details are stored in flat files in HDFS or ViewFS. Create a mapping that reads all the employee ID, name, and address details from the flat files in HDFS. The mapping must convert the denormalized data to hierarchical data and write it to a complex file target in HDFS or ViewFS.

You can use the target data for business analytics.

The following figure shows the example mapping:



You can use the following objects in the HDFS or ViewFS mapping:

HDFS Inputs

The inputs, Read_Address_Flat_File and Read_Name_Flat_File, are flat files stored in HDFS or ViewFS.

Data Processor Transformation

The Data Processor transformation, JSON_R2H_Denormalize_NameAndAddress, reads the flat files, denormalizes the data, and provides a binary, hierarchical output.

HDFS Output

The output, Write_Complex_File, is a complex file stored in HDFS or ViewFS.

When you run the mapping, the Data Integration Service reads the input flat files and passes the employee ID, name, and address data to the Data Processor transformation. The Data Processor transformation denormalizes the employee ID, name, and address data, and provides a hierarchical output in a binary stream. The binary and hierarchical output is written to the HDFS or ViewFS complex file target.

You can configure the mapping to run in a native or Hadoop run-time environment.

Complete the following tasks to configure the mapping:

1. Create an HDFS or ViewFS connection to read flat files from the Hadoop cluster.
2. Specify the read properties for the flat files.
3. Drag and drop the flat files into a mapping.
4. Create a Data Processor transformation. Set the Data Processor transformation port to binary.
5. Create an HDFS or ViewFS connection to write data to the complex file target.
6. Create a complex file data object write operation. Specify the following parameters:
   - The file as the resource in the data object.
   - The HDFS or ViewFS file location.
7. Drag and drop the complex file data object write operation into the mapping.

HDFS or View File System (ViewFS) Avro Read Mapping Example

Your organization needs to denormalize customer key, name, address, and other details. The customer details are stored in Avro files in HDFS or ViewFS. Import the Avro file object as a source. Create a mapping that reads all the customer details from the avro files in HDFS or ViewFS, and writes the customers details to an Oracle target.

You can use the target data for business analytics.

You can use the following objects in the HDFS or ViewFS mapping:

HDFS Inputs

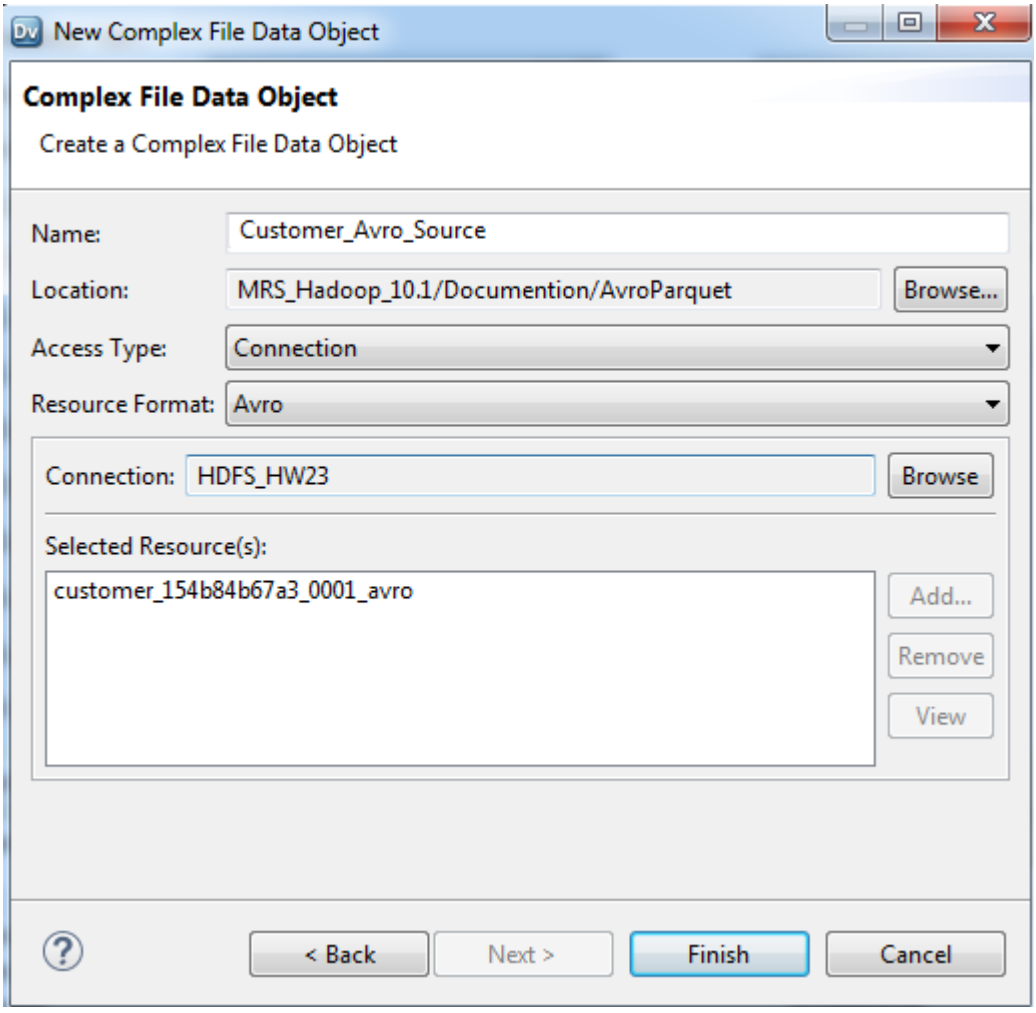
The Customer_Details_Avro file is an Avro files stored in HDFS or ViewFS.

HDFS Output

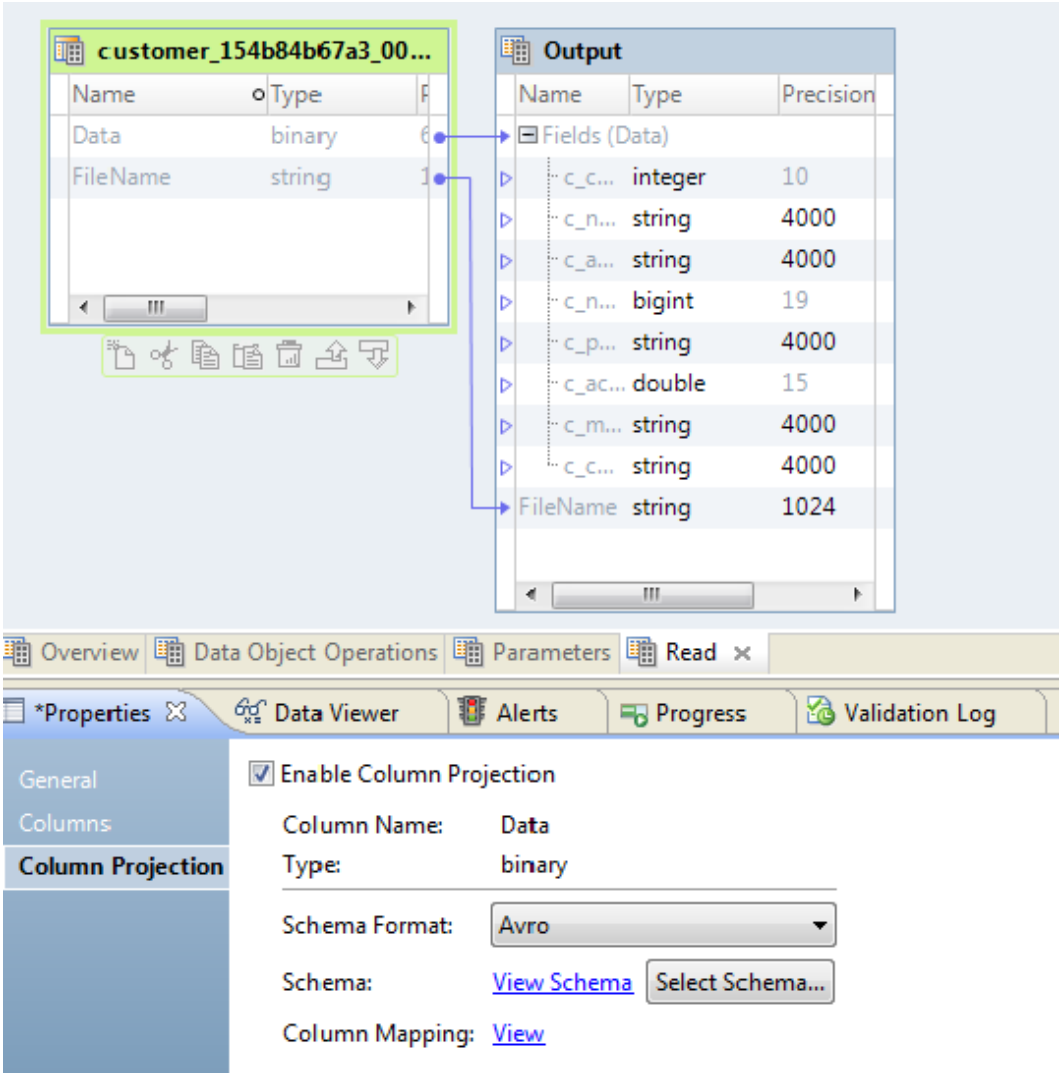
The Customer_Oracle_Target file is an Oracle object.

Create a Complex File Data Object

Create a complex file data object to read data from an Avro file. Verify that you select Avro as Resource Format. The following image shows the sample selection:

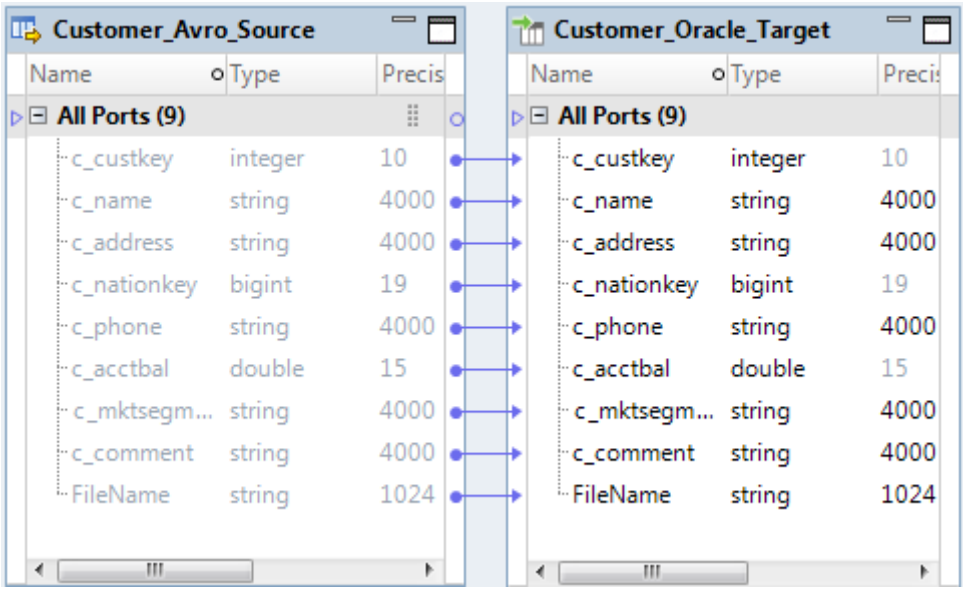


When you create the complex file data object, the read and write operations are created by default. You can view the columns present in the Avro file. The following image shows the sample data object read operation:



The Enable Column Projection is selected by default. You can view or update the associated schema and column mapping.

The following image shows the sample mapping:



When you run the mapping, the Data Integration Service reads the input Avro files and writes the hierarchical output directly to the Oracle target.

You can configure the mapping to run in a native or Hadoop run-time environment.

Perform the following tasks to configure the mapping:

1. Create an HDFS connection to read Avro file from the Hadoop cluster.
2. Create a complex file data object to import the Avro file. You must select Avro as Resource Format. Configure the read operation properties.
3. Create an Oracle database connection to write data to the Oracle target.
4. Create an Oracle data object and configure the write operation properties.
5. Drag the complex file data object read operation and Oracle data object write operation into the mapping.
6. Map ports and run the mapping.

APPENDIX A

Data Type Reference

This appendix includes the following topics:

- Data Type Reference Overview, 53
- Flat File and Transformation Data Types, 54
- Complex File and Transformation Data Types, 55
- Avro Data Types and Transformation Data Types, 55
- JSON Data Types and Transformation Data Types, 57
- ORC Data Types and Transformation Data Types, 57
- Parquet Data Types and Transformation Data Types, 59
- Rules and Guidelines for Data Types, 60

Data Type Reference Overview

Informatica Developer uses the following data types for HDFS data objects:

- Flat file data types. Flat file data types appear in the physical data object column properties.
- Complex file data types. Complex file data types appear in the physical data object column properties.
- Transformation data types. Set of data types that appear in the transformations. They are internal data types based on ANSI SQL-92 generic data types, which the Data Integration Service uses to move data across platforms. Transformation data types appear in all transformations in a mapping.

When the Data Integration Service reads source data, it converts the native data types to the comparable transformation data types before transforming the data. When the Data Integration Service writes to a target, it converts the transformation data types to the comparable native data types.

Flat File and Transformation Data Types

Flat file data types map to transformation data types that the Data Integration Service uses to move data across platforms.

The following table compares flat file data types to transformation data types:

Flat File Data type	Transformation Data type	Range
Bigint	Bigint	Precision of 19 digits, scale of 0
Datetime	Date/Time	Jan 1, 0001 A.D. to Dec 31, 9999 A.D. (precision to the nanosecond)
Double	Double	Precision of 15 digits
Int	Integer	-2,147,483,648 to 2,147,483,647
Nstring	String	1 to 104,857,600 characters
Number	Decimal	For transformations that support precision up to 38 digits, the precision is 1 to 38 digits, and the scale is 0 to 38. For transformations that support precision up to 28 digits, the precision is 1 to 28 digits, and the scale is 0 to 28. If you specify the precision greater than the maximum number of digits, the Data Integration Service converts decimal values to double in high precision mode. If the precision is greater than 15, the Data Integration Service converts decimal values to double in low-precision mode.
String	String	1 to 104,857,600 characters
TimestampWithTZ	TimestampWithTZ	Aug. 1, 1947 A.D to Dec. 31, 2040 A.D. -12:00 to +14:00 Precision of 36 and scale of 9. (precision to the nanosecond) Timestamp with Time Zone data type does not support the following time zone regions: - AFRICA_CAIRO - AFRICA_MONROVIA - EGYPT - AMERICA_MONTREAL **Note:** TimestampWithTZ is applicable only to the native environment.

When the Data Integration Service reads non-numeric data in a numeric column from a flat file, it drops the row and writes a message in the log. Also, when the Data Integration Service reads non-datetime data in a datetime column from a flat file, it drops the row and writes a message in the log.

Complex File and Transformation Data Types

Complex file data types map to transformation data types that the Data Integration Service uses to move data across platforms.

The following table lists the complex file data types that the Data Integration Service supports and the corresponding transformation data types:

Complex File Data Type	Transformation Data Type	Range and Description
Binary	Binary	1 to 104,857,600 bytes. You can read and write data of Binary data type in a Hadoop environment. You can use the user-defined functions to transform the binary data.

Avro Data Types and Transformation Data Types

Avro data types map to transformation data types that the Data Integration Service uses to move data across platforms.

The following table compares the Avro data types that the Data Integration Service supports and the corresponding transformation data types:

Avro Data Type	Transformation Data Type	Range
Array	Array	Unlimited number of characters.
Boolean	Integer	TRUE (1) or FALSE (0).
Bytes	Binary	Precision 4000.
Date	Date/Time	January 1, 0001 to December 31, 9999.
Decimal	Decimal	Decimal value with declared precision and scale. Scale must be less than or equal to precision. For transformations that support precision up to 38 digits, the precision is 1 to 38 digits, and the scale is 0 to 38. For transformations that support precision up to 28 digits, the precision is 1 to 28 digits, and the scale is 0 to 28. If you specify the precision greater than the maximum number of digits, the Data Integration Service converts decimal values to double in high precision mode.
Double	Double	Precision 15.
Fixed	Binary	1 to 104,857,600 bytes.
Float	Double	Precision 15.
Int	Integer	-2,147,483,648 to 2,147,483,647 Precision 10 and scale 0.

Avro Data Type	Transformation Data Type	Range
Long	Bigint	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807. Precision 19 and scale 0.
Map	Map	Unlimited number of characters.
Record	Struct	Unlimited number of characters.
String	String	1 to 104,857,600 characters.
Time	Date/Time	Time of the day. Precision to microsecond.
Timestamp	Date/Time	January 1, 0001 00:00:00 to December 31, 9999 23:59:59.997. Precision to microsecond.
Union	Corresponding data type in a union of ["primitive_type|complex_type", "null"] or ["null", "primitive_type|complex_type"].	Dependent on primitive or complex data type.

Avro Union Data Type

A union indicates that a field might have more than one data type. For example, a union might indicate that a field can be a string or a null. A union is represented as a JSON array containing the data types.
The Developer tool only interprets a union of ["primitive_type|complex_type", "null"] or ["null", "primitive_type|complex_type"]. The Avro data type converts to the corresponding transformation data type.

Avro Timestamp Data Type Support

The following table lists the Timestamp data type support for Avro file formats:

Timestamp Data type	Native	Spark
Timestamp_micros	Yes	Yes
Timestamp_millis	Yes	No
Time_millis	Yes	No
Time_micros	Yes	No

Unsupported Avro Data Types

The Developer tool does not support the following Avro data types:

- Enum
- Null
- Timestamp_tz

JSON Data Types and Transformation Data Types

JSON data types map to transformation data types that the Data Integration Service uses to move data across platforms.

The following table compares the JSON data types that the Data Integration Service supports and the corresponding transformation data types:

JSON	Transformation	Range
Array	Array	Unlimited number of characters.
Double	Double	Precision of 15 digits.
Integer	Integer	-2,147,483,648 to 2,147,483,647. Precision of 10, scale of 0.
Object	Struct	Unlimited number of characters.
String	String	1 to 104,857,600 characters.

Unsupported JSON Data Types

The Developer tool does not support the following JSON data types:

- Date
- Decimal
- Timestamp
- Enum
- Union

ORC Data Types and Transformation Data Types

ORC file data types map to transformation data types that the Data Integration Service uses to move data across platforms.

The following table lists the ORC file data types that the Data Integration Service supports and the corresponding transformation data types:

ORC File Data Type	Transformation Data Type	Range and Description
BigInt	BigInt	-9223372036854775808 to 9,223,372,036,854,775,807.
Boolean	Integer	TRUE (1) or FALSE (0).
Char	String	1 to 104,857,600 characters.

ORC File Data Type	Transformation Data Type	Range and Description
Date	Date/Time	January 1, 0001 to December 31, 9999.
Double	Double	Precision of 15 digits.
Float	Double	Precision of 15 digits.
Integer	Integer	-2,147,483,648 to 2,147,483,647.
SmallInt	Integer	-32,768 to 32,767.
String	String	1 to 104,857,600 characters.
Timestamp	Date/Time	January 1, 0001 00:00:00 to December 31, 9999 23:59:59.997. Precision to microsecond.
TinyInt	Integer	-128 to 127.
Varchar	String	1 to 104,857,600 characters.

When you run a mapping on the Spark or Databricks Spark engine to write an ORC file to a target, the Data Integration Service writes the data of the Char and Varchar data types as String.

Note: You can use ORC data types to read and write complex file objects in mappings that run on the Spark engine only.

Unsupported ORC Data Types

The Developer tool does not support the following ORC data types:

- Map
- List
- Struct
- Union

Parquet Data Types and Transformation Data Types

Parquet data types map to transformation data types that the Data Integration Service uses to move data across platforms.

The following table compares the Parquet data types that the Data Integration Service supports and the corresponding transformation data types:

Parquet	Transformation	Range
Binary	Binary	1 to 104,857,600 bytes
Binary (UTF8)	String	1 to 104,857,600 characters
Boolean	Integer	-2,147,483,648 to 2,147,483,647 Precision of 10, scale of 0
Date	Date/Time	January 1, 0001 to December 31, 9999.
Decimal	Decimal	Decimal value with declared precision and scale. Scale must be less than or equal to precision. For transformations that support precision up to 38 digits, the precision is 1 to 38 digits, and the scale is 0 to 38. For transformations that support precision up to 28 digits, the precision is 1 to 28 digits, and the scale is 0 to 28. If you specify the precision greater than the maximum number of digits, the Data Integration Service converts decimal values to double in high precision mode.
Double	Double	Precision of 15 digits.
Float	Double	Precision of 15 digits.
Int32	Integer	-2,147,483,648 to 2,147,483,647 Precision of 10, scale of 0
Int64	Bigint	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807 Precision of 19, scale of 0
Map	Map	Unlimited number of characters.
Struct	Struct	Unlimited number of characters.
Time	Date/Time	Time of the day. Precision to microsecond.

Parquet	Transformation	Range
Timestamp	Date/Time	January 1, 0001 00:00:00 to December 31, 9999 23:59:59.997. Precision to microsecond.
group (LIST)	Array	Unlimited number of characters.

The Parquet schema that you specify to read or write a Parquet file must be in smaller case. Parquet does not support case-sensitive schema.

Parquet Timestamp Data Type Support

The following table lists the Timestamp data type support for Parquet file formats:

Timestamp Data type	Native	Spark
Timestamp_micros	Yes	No
Timestamp_millis	Yes	No
Time_millis	Yes	No
Time_micros	Yes	No
int96	Yes	Yes

Unsupported Parquet Data Types

The Developer tool does not support the following Parquet data types:

- Timestamp_nanos
- Time_nanos
- Timestamp_tz

Rules and Guidelines for Data Types

Some data types for complex files might be applicable only when you use specific Hadoop distributions.

Before you read from or write to complex files, consider certain rules and guidelines for the data types.

Decimal data type

To process Decimal data types with precision up to 38 digits on the Data Integration Service, set the `EnableSDKDecimal38` custom property to `true` for the Data Integration Service.

Avro

You can process Date, Decimal, and Timestamp data types from Avro files in mappings that run on the Data Integration Service or on the Spark engine in the Cloudera CDP distribution. You can process Time data types only on the Data Integration Service.

JSON

You can read and write complex file objects in JSON format only on the Spark engine.

Parquet

The following rules apply for Parquet files:

- If the Parquet files contain Decimal data types, you can process the mapping on the Data Integration Service or on the Spark engine in Cloudera CDP, Amazon EMR, MapR, and Azure HDInsight HDI distributions.
- When you import a Parquet file, the format of the schema for the String data type differs based on the distribution. For Cloudera CDP, the schema for String appears as UTF8, while for other distributions, it appears as UTF8.
  For example, when you use Cloudera CDP, the schema for String appears as: `optional binary c_name (UTF8);`

  In other distributions, String appears as: `optional binary c_name (STRING);`
- Consider the following guidelines for Date and Time data types:
  - The Data Integration Service, Blaze, and Spark engine in the Azure HDInsight HDI and Cloudera CDP distributions can process Date, Time, and Timestamp data types till microseconds. The Data Integration Service or Spark engine in the MapR distribution can process Date data types and the Time and Timestamp data types till milliseconds.
  - When the Data Integration Service reads the Date data type that does not have a time value, it adds the time value, based on the time zone, to the date in the target.
    For example, if the source contains the Date value `1980-01-09 00:00:00`, the following incorrect Time value is generated in the target: `1980-01-09 05:30:00`
  - When the Data Integration Service reads the Time data type, it writes incorrect date values to the target.
    For example, if the source contains the Time value `1980-01-09 06:56:01.365235000`, the following incorrect Date value is generated in the target: `1899-12-31 06:56:01.365235000`

INDEX

A

Avro data types
  transformation data types 55

C

complex file data object read operation
  creation 23
complex file data objects
  creating 21
  general properties 19
  objects properties 19
  overview 19
complex file output
  parsing 21
complex file read properties
  advanced properties 30
  general properties 27
  overview 26
  ports properties 28
  schema properties 28
  sources properties 29
complex file write properties
  advanced properties 38
  general properties 36
  overview 35
  ports properties 36
  schema properties 36
  target properties 37
complex files
  compression 20
  decompression 20
  input formats for text files 26
  output collection mode 34
  partitioning 26
  streaming 33
custom formats
  configuration 24
  overview 24

D

Data Processor transformation
  configuration 21
data type reference
  complex files 55
  flat files 54
data Type reference
  overview 53

F

File Naming Convention 18
FileName port 14
flat file data objects
  compression 17
  configuring an HDFS connection 18
  decompression 17
  partitioning 16
  read properties 25
  rules and guidelines for using 17
  write properties 32

H

HDFS connections
  creating 12
  overview 10
  properties 10
HDFS data objects
  complex file data objects 18
  flat file data objects 16
  overview 13
HDFS mappings
  avro data read example 50
  data extraction example 47
  data load example 48
  overview 40

J

JSON data types
  transformation data types 57

M

mapping run-time environment
  Hadoop 46

O

ORC file data types
  transformation data types 57

P

Parquet data types
  transformation data types 59
PowerExchange for HDFS
  data extraction 25
  data load 32

PowerExchange for HDFS *(continued)*
overview 8
PowerExchange for HDFS configuration
overview 9
prerequisites 9

R

rules and guidelines
complex file data object operation 23
rules and guidelines *(continued)*
FileName port 14

W

working with FileName port 14